

{Block1}

{CH=1}

***Web Server:** Web Server is a server software that handles HTTP requests and responses to deliver web pages or web content to clients (i.e. web browser) using HTTP protocol. Web browser communicates with web server using the Hypertext Transfer Protocol (HTTP). Hypertext Transfer Protocol (HTTP) is specially meant to communicate between Client and Server using Web (or Internet). To successfully execute web application, the number of server side technologies (such as JSP, Servlets and PHP) and their libraries are installed on the web server. Without these libraries, a web server cannot execute those server technologies based applications. An example of web server is Apache HTTP Server

***Web Container:** Web Container is a web server component that handles Servlets, Java Server Pages (JSP) files, and other Web-tier components. Web container is also called a Servlet Container or Servlet Engine. It is the responsibility of the Web container to map a URL to a particular servlet. Also, Web container ensures that the mapped URL requester has the correct access rights. It means that it provides the run time environment to web applications. The most common web containers are Glassfish, Eclipse, JBOSS, Apache Tomcat, WebSphere and Web Logic. For deployment and running the JSPs/Servlets we need a compatible web server with a servlet container.

***J2EE (Java 2 Enterprise Edition):** The J2EE platform is a set of services, application programming interfaces (APIs) and protocols. J2EE is used to develop and deploy multi-tier web-based enterprise applications using a series of protocols and application programming interfaces (APIs). J2EE contains several APIs such as Java Servlets, Java Server Pages (JSP), Enterprise Java Beans (EJB), Java Database Connectivity (JDBC), Java Message Service (JMS), Java Naming and Directory Interface (JNDI) and so on. The J2EE application model divides applications into three basic parts like components, containers and connectors. The application developers works on the components part, whereas system vendors are responsible for implementing containers and connectors. Containers act as a mediator between clients and components by providing services like transaction support and resource pooling. Connectors provide bidirectional communication between J2EE components and enterprise systems.

A J2EE application contains four components or tiers: Presentation, Application, Business, and Resource adapter components. The presentation component is the client side component that is visible to the client and runs on the client's server. The Application component is web side layer that runs on the J2EE server. The business component is the business layer which includes server-side business logic such as JavaBeans, and it is also run on the J2EE server. The resource adaptor component comprises an enterprise information system.

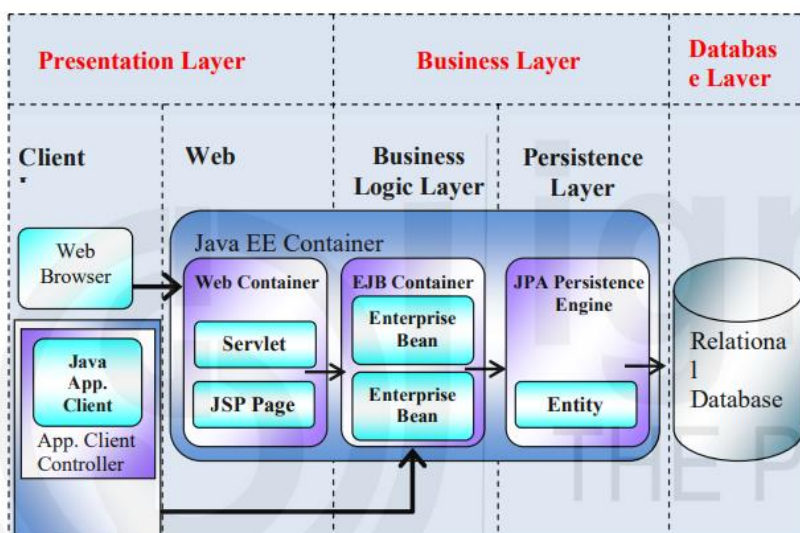


Figure 1: J2EE Architecture

Q What is J2EE ? Briefly explain J2EE architecture. 10

Q short note on J2EE Architecture (5)

*DESIGN PATTERNS:

Q What is a design pattern? Explain the advantages of using a design pattern in enterprise applications development, with the help of an example. 10 (Repeat)

Design patterns are the best solutions that are tested, verified developed prototypes that can speed up your development process. When you may face problems during software development, the design patterns gives you explanations for those problems. The concept of design pattern has been initiated by four authors collectively known as GOF (Gang of Four). Design Patterns provides a general reusable solution to commonly occurring problems. It is not a complete design that can be directly mapped into your code and solve your purpose. There are 23 classic design patterns and they are categorized into three groups: Creational, Structural and behavioral.

1) **Creational Design Pattern:** The Creational design patterns provide a way to deal with the creation of an object and define 5 design patterns such as Abstract Factory, Builder, Factory, Prototype and Singleton pattern.

- Abstract Factory design pattern allows to create families of related objects. This pattern permits us to create a Factory for factory classes.
- The Builder design pattern provides a valuable solution to many object creation problems in object-oriented programming.
- The Prototype design pattern allows us to produce new objects

2) **Structural design pattern:** The Structural design patterns are related to the creation of class and object structure. These patterns define seven design patterns: Adapter, Bridge, Composite, Decorator, Façade, flyweight, and Proxy.

3) **Behavioral design patterns:** The third design patterns category is Behavioral, which defines 11 design patterns: Chain of responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, and Visitor. They are specifically concerned with the interaction between objects.

Benefits of Using Design Pattern:

- Speed up the development process by providing tested, proven development paradigms.
- Reusing design Patterns helps to prevent issues that can cause major problems and it also improves code readability for coders and architects who are familiar with the patterns.
- Design Pattern provide general solutions, documented in a format that doesn't require specifics tied to a particular problem.
- Design Pattern allow developers to communicate using well known, well understood names for software Interaction

*MVC Design Pattern

Model View Controller (MVC) design pattern is an architectural pattern for web applications. It can be used across many frameworks with many programming languages such as Java, PHP, Python, Ruby, etc. The MVC design pattern described three layers such as Model, View and Controller. In MVC pattern, M (Model) denotes only the pure application data and does not contain any logic for representing data to a user, whereas V (View) is responsible for presenting data to the user. The C (Controller) comes between the view and the model layer and it is responsible for accepting a request from the user, performs interactions on the data model objects, and sends it back to the view layer. **The UML diagram of MVC design pattern is depicted in figure:**

The example contains total of four java files, from which three java files for MVC layers and one java file containing the main() method. For defining the MVC pattern, the first java file is created as 'UniversityModel', which acts as a model, 'univView' as a view that can display university details and 'UnivController' file is responsible for storing the data in univ object and updateView() method updates the data in a 'univView' class method.

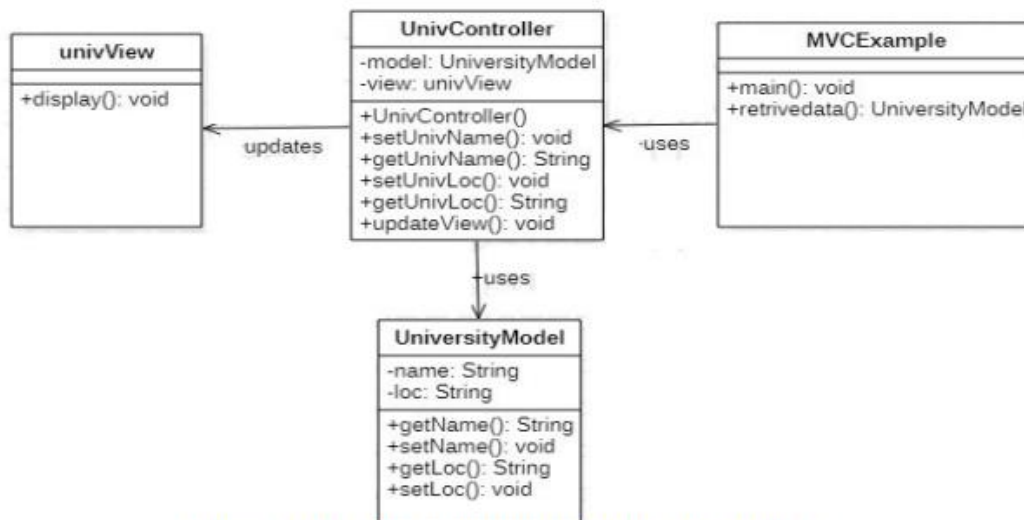


Figure 2: UML class diagram of MVC design pattern is

*Repository Design Pattern:

In plain words, repository means something is deposited in storage. The repository meaning is same as in terms of design pattern, it is related to the data. The repository design pattern is used in data-centric applications. The Repository pattern is used to isolate the business logic layer and the data source layers in your application. The repository layer comes between the domain and data mapping layers. As shown in figure 4, the repository resides in the mid of the client business logic and data source layer. At the data source layer, a database can be kept/used.

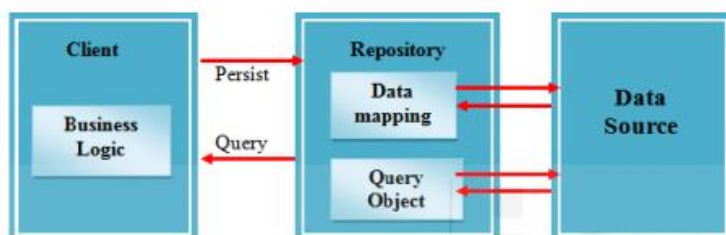


Figure 4: Block diagram for Repository Design Pattern

Suppose client business logic wishes a query from data source then it first goes to repository layer thereafter repository will send the query to the database, and then repository maps the data to the business entity. Similarly, when client business logic wants to save data in the database, data will not directly go to the database but will go through the repository layer

Singleton Design pattern: The singleton pattern is a simplest software design pattern amongst the 23 well-known "Gang of Four" design patterns and it comes under the creational design pattern category, which deals with the creation of an object. As the name suggests, Singleton design pattern is used to create only one instance of a class, and all other classes of the application can use this instance. This type of pattern is mostly used in database connection and multi-threaded applications. Singleton pattern is used in logging, caching, thread pools, configuration settings etc. You can use Singleton pattern in other design patterns like Abstract Factory, Builder, Facade, Prototype etc., while developing your own solutions to a specific problem.

The UML class diagram of Singleton pattern:

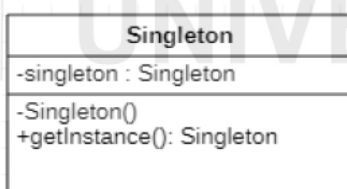


Figure 3: UML class diagram of Singleton pattern

Factory Design Pattern

A Factory Design Pattern or Factory Method Pattern is one of the most used design pattern in java. It is a creational design pattern which deals with the creation of an object. As per the GOF, this pattern defines an interface or abstract class for creating an object, but subclasses are responsible for creating the instance of the class. In other words, this pattern is used to create an object where object creation logic is hidden to the client and refers to the newly created object using a common interface. This design pattern is also known as 'Virtual Constructor'.

To implement the Factory Design Pattern, first, you define a factory method inside an interface or abstract class and let the subclass use the above factory method and decide which object to create.

Advantage of Factory Design Pattern

- Factory Method Pattern allows the sub-classes to choose the type of objects to create.
 - It promotes the **loose-coupling** by eliminating the need to bind application-specific classes into the code. That means the code interacts solely with the resultant interface or abstract class, so that it will work with any classes that implement that interface or that extends that abstract
-

{CH=2}

SERVLETS

Java Servlets are small, platform-independent java programs that run on the Java-enabled web server. It extends from a Java class or rather interface and requires you to implement certain methods so that web container or servlet container (Tomcat, etc.) is able to send execution to your servlet. Basically it creates a class that extends either GenericServlet or HttpServlet, overriding the appropriate methods, so it handles requests.

Servlet extends the capabilities of web servers that host applications accessed by means of a request-response programming model. For developing web applications, Java Servlet technology defines HTTP-specific servlet classes. A HTTP Servlet runs under the HTTP protocol. This protocol is an asymmetrical request-response protocol where the client sends a request message to the server, and the server returns a response for requested data.

Servlets follows client-server architecture in which client (browser) sends data to server and it requires some protocol (set of rules) for communication between the two. Hypertext Transfer Protocol (HTTP) is specially meant to communicate between Client and Server using Web (or Internet).

HTTP: HTTP Protocol is a network protocol used for transferring files on the Internet . HTTP is the primary protocol used for most web applications on the Internet. Whether the application are written in Java, ASP.Net or PHP; all web applications use HTTP. This protocol is the foundation of any data exchange on the Web. HTTP works based on request-response model between a client and the server. An HTTP client sends a request message to a HTTP server. The server returns a response message to the client. As HTTP is a stateless protocol, the current request does not know what has been done in the previous requests.

HTTP is also called connectionless protocol, which means that the client establishes a connection with the server before sending a request, and the server sends back response to the client over this connection only.

HTTP was initially developed by Tim Berners-Lee and his team in early 1990. HTTP/1.1 defines seven HTTP methods GET, POST, HEAD, OPTIONS, TRACE, PUT, DELETE

HTTP messages manage the information exchange between a server and a client. There are two types of HTTP messages: **HTTP requests** sent by the client to the server, and **HTTP responses**, the answer from the server.

HTTP Requests

Whenever a client sends a message to a server it is an HTTP request. when the client sends a request to the server, the server returns a response to the client. Web Client sends a request specifying one of the seven HTTP request methods, the location of the resource to be invoked, protocol version, a set of optional headers and an optional message body.

Some Http Servlet Request Methods

Meth od	description
getMethod()	Returns the HTTP requ est method
getRequest URI()	Returns the complete requested URI re
getProtocol()	Returns the protocol used by the browser
getPathInfo()	only returns the path passed to the servlet
getRemote ddr()	Returns the IP address of the client

HTTP Response

Once a server receives a request, it interprets the request message and responds with an HTTP response message. The server sends back a response to the client containing the version of HTTP we are using, a response or status code, a description of the response code, a set of optional headers, and an optional message body.

HTTP response status codes is used to indicate whether a specific HTTP request has been successfully completed or not. Responses are grouped in five classes:

HTTP Response Classes

S.No.	Status Code	Description
1	Informational responses (100–199)	Request has been received and the process is continuing.
2	Successful responses (200–299)	Action was successfully received, understood, and accepted.
3	Redirects (300–399)	Action must be taken to complete the request.
4	Client errors (400–499)	Request contains incorrect syntax or cannot be fulfilled - It means the server failed to fulfil an apparently valid request.
5	Server errors (500–599)	Server failed to fulfil a valid request.

The GET Method

GET is used to request data from a specified resource. Note that the query string (name/value pairs) is sent in the URL of a GET request:

Some notes on GET requests:

- GET requests can be cached
- GET requests remain in the browser history
- GET requests can be bookmarked
- GET requests should never be used when dealing with sensitive data
- GET requests have length restrictions
- GET requests are only used to request data (not modify)

The POST Method

POST is used to send data to a server to create/update a resource. The data sent to the server with POST is stored in the request body of the HTTP request: name1=value1&name2=value2

Some notes on POST requests:

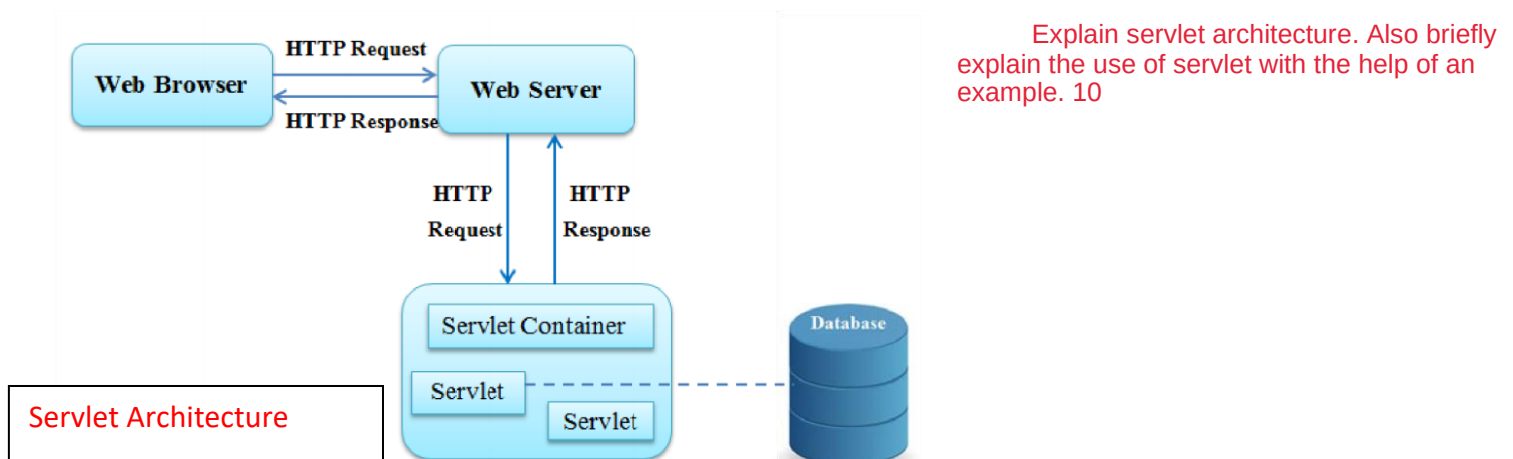
- POST requests are never cached
- POST requests do not remain in the browser history
- POST requests cannot be bookmarked
- POST requests have no restrictions on data length

The HEAD Method

HEAD is almost identical to GET, but without the response body. In other words, if GET /users returns a list of users, then HEAD /users will make the same request but will not return the list of users.

HEAD requests are useful for checking what a GET request will return before actually making a GET request - like before downloading a large file or response body

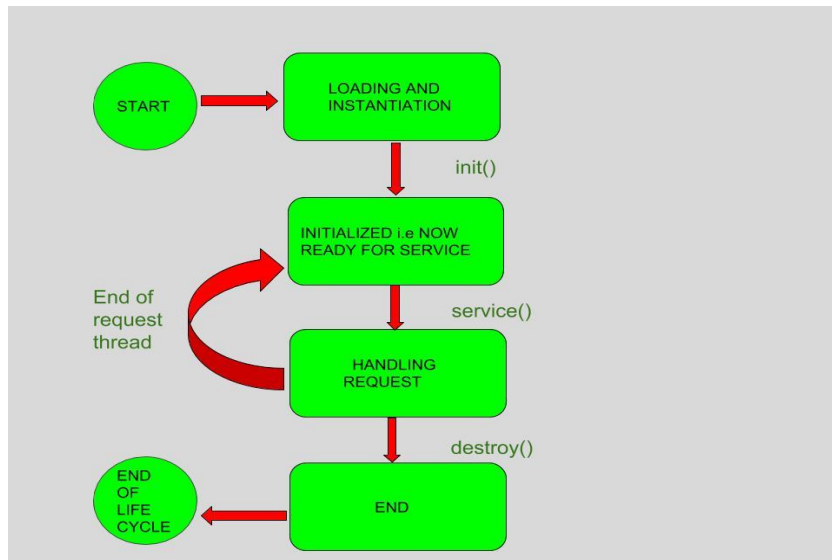
SERVLET ARCHITECTURE



Servlets are the Java program that runs on the Java-enabled web server. The client or web browser sends the HTTP request to the web server. The web server receives the request and passes the request to the servlet container and subsequently the servlet container forwards this request to the corresponding servlet. The servlet processes the request and generates the response in the form of output. This process may require communication to a database and may also invoke a web service, or computing the response directly. After processing, the servlet builds the response object and sends it back to the Web Container. The servlet container sends the response back to the web server; the web server sends the response back to the browser and the browser displays it on the screen. The diagram shows the execution of servlet.

We need to use Servlet API to create servlets. These APIs allow developer to build programs that can run with a web server. The Servlet APIs (Application Programming Interface) contains two packages: javax.servlet and javax.servlet.http. These two packages make up the servlet architecture. The javax.servlet and javax.servlet.http packages provide interfaces and classes for writing servlets. The javax.servlet package contains generic interfaces and classes that are implemented and extended by all the servlet. The javax.servlet.http package contains a number of classes and interfaces that are used to create HTTP protocol-specific Servlets.

SERVLET LIFE CYCLE



What is Servlet Programming Briefly explain servlet life cycle. Also write a servlet which displays “welcome to web programming.” 10

The entire life cycle of a Servlet is managed by the **Servlet container** which uses the **javax.servlet.Servlet** interface to understand the Servlet object and manage it.

Stages of the Servlet Life Cycle: The Servlet life cycle mainly goes through four stages,

- Loading a Servlet.
- Initializing the Servlet.
- Request handling.
- Destroying the Servlet.

1. Loading a Servlet: The classloader is responsible to load the servlet class. The servlet class is loaded when the first request for the servlet is received by the web container.

- **Loading :** Loads the Servlet class.
- **Instantiation :** Creates an instance of the Servlet. To create a new instance of the Servlet, the container uses the no-argument constructor.

2. Initializing a Servlet: The web container creates the instance of a servlet after loading the servlet class. The servlet instance is created only once in the servlet life cycle.

init method is invoked: The web container calls the init method only once after creating the servlet instance. The init method is used to initialize the servlet. It is the life cycle method of the javax.servlet.Servlet interface. Syntax of the init method is given below:

public void init(ServletConfig config) **throws** ServletException

3. Handling request: After initialization, the Servlet instance is ready to serve the client requests. The Servlet container performs the following operations when the Servlet instance is located to service a request :

- It creates the **ServletRequest** and **ServletResponse** objects. In this case, if this is a HTTP request, then the Web container creates **HttpServletRequest** and **HttpServletResponse** objects which are subtypes of the **ServletRequest** and **ServletResponse** objects respectively.
- After creating the request and response objects it invokes the `Servlet.service(ServletRequest, ServletResponse)` method by passing the request and response objects.

service method is invoked: The `service()` method is used to handle incoming requests and generate a response. This method is called by the servlet container and cannot start servicing the request until the servlet's `init()` method has been executed. The most common use of this method is in the `HttpServlet` class. The syntax of the service method of the `Servlet` interface is given below:

```
public void service(ServletRequest request, ServletResponse response)
    throws ServletException, IOException
```

- 4. Destroying a Servlet:** When a Servlet container decides to destroy the Servlet, it performs the following operations,
- It allows all the threads currently running in the service method of the Servlet instance to complete their jobs and get released.
 - After currently running threads have completed their jobs, the Servlet container calls the **destroy()** method on the Servlet instance.

destroy method is invoked: The `destroy()` method is called only once when all the processing of the servlet is over and it is at the end of its life cycle. When your application is stopped or Servlet Container shuts down, this method will be called to close down those shared resources and other clean up required before the servlet is taken out of service. This is the place where any resources that were created in `init()` method will be cleared up.

```
public void destroy()
```

Creating a Web Servlet to display “welcome to web programming.”

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.util.*;

public class WelcomeServlet extends HttpServlet
{
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException,
    IOException
    {
        PrintWriter out = response.getWriter();
        System.out.println("<HTML>");
        System.out.println("<HEAD>");
        System.out.println("<TITLE>Servlet Testing</TITLE>");
        System.out.println("</HEAD>");
        System.out.println("<BODY>");
        System.out.println("Welcome to the IGNOU Family!!");
        System.out.println("</BODY></HTML>");
    }
}
```

SESSION MANAGEMENT

short note on Session management (5)

Session: The server should recognize a series of requests from the same user which form a single working 'session'. For example, in net banking application, each client can be differentiated from another client by associating a unique identifier in request and response within a specific working session.

State: The server should be able to remember information related to the previous request and other business transaction that are made for that request. For example, in net banking application, state comprises client information such as account number and amount transaction made within the particular session.

Session management does not change the nature of HTTP protocol i.e., stateless feature provides a way to remember the client information. Session management is also known as session tracking, which permits Servlet/JSP to maintain information about a series of request from the same client. It is a mechanism to store session information for each client. For example, When a user visits any web application, unique identification information about the user is stored in an object available during the particular session until the user quits the web application.

There are four ways to manage sessions: HttpSession object, Cookies, URL rewriting, and hidden fields.

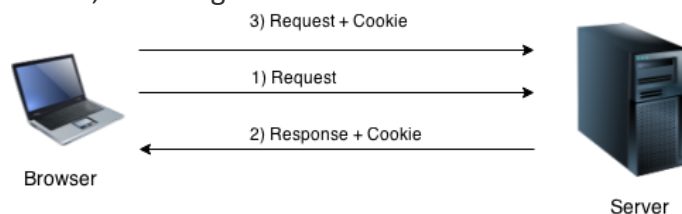
1.) HttpSession Object : A user session is represented by the HttpSession object. A session is established between an HTTP client and an HTTP server using the HttpSession interface. A user session is a collection of data about a user that spans many HTTP requests. The request must be made. Before sending any document content to the client, you must first call getSession().

```
HttpSession session = request.getSession( );
```

```
Session.setAttribute("username", "password");
```

2.) Cookies: A **cookie** is a small piece of information that is persisted between the multiple client requests. A cookie has a name, a single value, and optional attributes such as a comment, path and domain qualifiers, a maximum age, and a version number.

By default, each request is considered as a new request. In cookies technique, we add cookie with response from the servlet. So cookie is stored in the cache of the browser. After that if request is sent by the user, cookie is added with request by default. Thus, we recognize the user as the old user.



Types of Cookie

There are 2 types of cookies in servlets.

1. Non-persistent cookie
2. Persistent cookie

Non-persistent cookie: It is **valid for single session** only. It is removed each time when user closes the browser.

Persistent cookie: It is **valid for multiple session**. It is not removed each time when user closes the browser. It is removed only if user logout or signout.

Advantage of Cookies

1. Simplest technique of maintaining the state.
2. Cookies are maintained at client side.

Disadvantage of Cookies

1. It will not work if cookie is disabled from the browser.
2. Only textual information can be set in Cookie object.

3.)URL REWRITING: In URL rewriting, we append a token or identifier to the URL of the next Servlet or the next resource. The token (parameter) is embedded in each URL. When client submits request using such URLs, the token is retransmitted to the server. In each dynamically generated page, server embeds an extra query parameter or extra path information. If a browser does not support cookies then in that case URL rewriting technique is the best alternative for session management.

Advantage of URL Rewriting

1. It will always work whether cookie is disabled or not (browser independent).
2. Extra form submission is not required on each pages.

Disadvantage of URL Rewriting

1. It will work only with links.
2. It can send Only textual information.

4.)Hidden Field: The information is inserted into the web pages via the hidden form field, which is then transferred to the server. These fields are hidden from the user's view. When client submits the form, additional fields will also be sent in the request in the form of hidden fields to keep track of the session.

```
<input type = hidden' name = 'session' value = '12345' >
```

Advantage of Hidden Form Field

1. It will always work whether cookie is disabled or not.

Disadvantage of Hidden Form Field

1. It is maintained at server side.
2. Extra form submission is required on each pages.
3. Only textual information can be used.

SERVLET COLLABORATION

When two or more servlets can communicate or share common information, it is Servlet Communication or Servlet Collaboration. Servlet collaboration means sharing information among the servlets. This requires each servlet to know the other servlet with which it collaborates. This enables passing/sharing information from one servlet to the other through method invocations.

Java provides Servlet-API to accomplish Servlet Collaboration and Servlet-API covers two interfaces namely: javax.servlet.RequestDispatcher and javax.servlet.http.HttpServletResponse. Both interfaces have various methods which are used for sharing information between servlets.

1.) Using RequestDispatcher Interface: The RequestDispatcher interface provides the facility of dispatching the request to another resource it may be html, servlet or jsp. This interface can also be used to include the content of another resource also. It is one of the way of servlet collaboration. It is found in javax.servlet package. There are two methods defined in the RequestDispatcher interface. forward () and include() for dispatching requests to/from web resources.

- **forward() Method** : This method is used to forward a request from a servlet to another web resource like servlet, JSP file or HTML file on the server. When this method is called, control is transferred from the current to the next resource called. The signature of this method is as follows:

```
public void forward(ServletRequest request, ServletResponse response) throws ServletException,  
java.io.IOException
```

- **include() method** : This method is used to include the content of another servlet, JSP page, HTML file in the servlet response. After calling this method, the response of another resource is included in the called resource. The signature of this method is as follows:

```
public void include(ServletRequest request, ServletResponse response) throws ServletException,  
java.io.IOException
```

2.) Using HttpServletResponse Interface:

The HttpServletResponse Interface captures the functionality of a response object that is returned to the client from an HTTP servlet. This interface offers many protocol-specific methods which are not available in ServletResponse interface. This interface extends the javax.servlet.ServletResponse interface. Two methods are used when sending output to the browser.

- **setContentTypes(java.lang.String type):** This method is used to set the type of the response data e.g. text, HTML etc.
`res.setContentType("text/html");`
- **getWriter():** This method returns the PrintWriter object for sending text data to the client.
`PrintWriter out = res.getWriter();`
- **sendRedirect():** This method of HttpServletResponse interface is used to redirect response to another web resource such as HTML, servlet or jsp file. This method works at the client side. This method always sends a new request. It can be used within and outside the application. For some applications, you want to send the request outside your web application and then it becomes most useful because it takes more time to fetch resources.

The syntax of sendRedirect() method is as under:

```
public void sendRedirect(String URL) throws IOException;
```

3.) Using ServletContext Interface:

In servlet programming, servlet context is the environment where the servlet runs. The ServletContext's object is created by the web container at the time of deploying the project. The ServletContext object is used to get configuration information from deployment descriptor file (web.xml), which will be available to any servlet or JSPs that are component of the 'webapps'. The ServletContext Interface defines various methods which are used by servlets to communicate with its servlet container.

- **setAttribute() Method :** This method stores an object and binds the object with the given attribute name in the application scope. It means that the said object is accessible from any servlet within the same web application. If attribute name already exists in the ServletContext, the old bound object will be replaced by the object passed to this method.
`void setAttribute(java.lang.String name, java.lang.Object obj)`
- **getAttribute() Method :** This method is used to get attribute with the given name from context. It returns the value of named attribute as an Object or NULL if there is no attribute by that name.
`Object getAttribute(String name)`

{CH=4}

JSP(JAVA SERVER PAGE)

- It stands for **Java Server Pages**.
- It is a server side technology.
- It is used for creating web application.
- It is used to create dynamic web content.
- In this JSP tags are used to insert JAVA code into HTML pages.
- It is an advanced version of Servlet Technology.
- It is a Web based technology helps us to create dynamic and platform-independent web pages.
- In this, Java code can be inserted in HTML/ XML pages or both.
- JSP is first converted into servlet by JSP container before processing the client's request.

JSP Life Cycle: The life cycle of JSP page is controlled by three methods i.e. `jspInit()`, `jspService()` and `jspDestroy()`.

- **jspInit()** : The `jspInit()` method is called only once during life cycle of a JSP. Similarly, servlet also has an `init()` method whose purpose is the same as that of `jspInit()`. `jspInit()` method is used to initialize objects and variables that are used throughout the life cycle of JSP. This method is defined in `JspPage` interface. This method is invoked when the JSP page is initialized. It has no parameters, returns no value and thrown no exceptions. The signature of the method is as follows:

```
public void jspInit() { // Initialization code }
```

- **_jspService()** : `_jspService()` is the method that is called every time the JSP page is requested to serve a request. This method is defined in the `javax.servlet.jsp.HttpJspPage` interface. This method takes `HttpServletRequest` and `HttpServletResponse` objects as an argument. The `_jspService()` method corresponds to the body of the JSP page. It is defined automatically by the processor and should never be redefined by you. It returns no value. The underscore ('_') signifies that you cannot override this method. The signature of the method is as follows:

```
public void _jspService  
(  
    javax.servlet.http.HttpServletRequest request, javax.servlet.http.HttpServletResponse response  
) throws javax.servlet.ServletException, java.io.IOException {  
    // services handling code  
}
```

- **jspDestroy()**- The `jspDestroy()` is invoked only once when the JSP page is about to be terminated. It is synonymous to the `destroy()` method of a servlet. You have to override `jspDestroy()` if you need to perform any cleanup, such as releasing database connections or closing open files. The signature of the method is as follows:

```
public void jspDestroy() { // cleanup code }
```

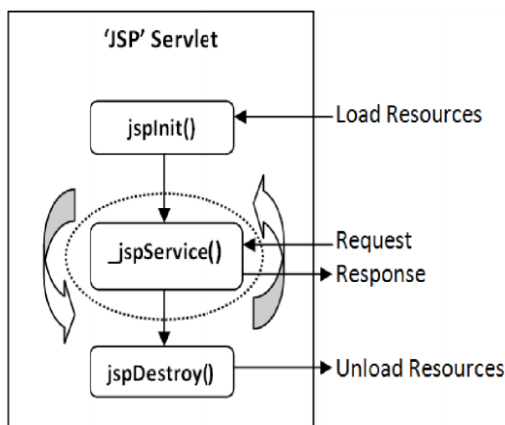


Figure 9: life cycle of JSP Page

JSP API The JSP technology is based on JSP API (Application Programming Interface) that contains two packages such as **javax.servlet.jsp** and **javax.servlet.jsp.tagext**. In addition to these two packages, JSP also needs two packages of servlets such as **javax.servlet** and **javax.servlet.http**. Both packages provide interfaces and classes for writing servlets.

Package `javax.servlet` has 2 interfaces and 4 classes.

Interface	Description
-----------	-------------

JspPage	The JspPage is the interface that all JSP servlet classes must implement. The JspPage interface has two methods, JspInit and JspDestroy. The jspInit method is similar to the 'init' method in the javax.servlet.Servlet interface. The jspDestroy method is analogous with destroy method of the javax.servlet.Servlet interface. You have studied both the methods in section 4.3 of this Unit. JSP developers rarely make full use of these two methods.
HttpJspPage	The HttpJspPage interface directly extends the JspPage interface. Each JSP page implements this interface. It supports only one method _jspService(). The _jspService() method is already described in life cycle of JSP in section 4.3 of this Unit.
Class	Description
JspEngineInfo	The JspEngineInfo is an abstract class that provides information of the current JSP engine.
JspFactory	The JspFactory is an abstract class that defines a number of factory methods available to a JSP page at runtime for the purposes of creating instances of various interfaces and classes used to support the JSP implementation.
JspWriter	The jspWriter class is derived from the java.io.Writer class. This is the normal mechanism for JSP pages to produce output to the response. No. of methods are defined in this class such as print() and println().
PageContext	The PageContext class is an abstract class. It extends JspContext to provide useful context information when JSP technology is used in a servlet environment. This class provides methods that are used to create other objects. For example, getServletConfig() returns a ServletConfig object and getServletContext() returns a ServletContext object.

Components of JSP

1.) Directives:

What is JSP ? What are JSP directives ? Explain use of any two JSP with help of program. 10

It guides the JSP container for translating and compilation of JSP page. Directives control the processing of an entire JSP page. It appears at the top of the page. Using directives, the container translates a JSP page into the corresponding servlet. They do not directly produce any output. A directive component comprises one or more attributes name/value pairs

A directive tag always appears at the top of your JSP file. It is global definition sent to the JSP engine. Directives contain special processing instructions for the web container. You can import packages, define error handling pages or the session information of the JSP page. Directives are defined by using `<%@` and `%>` tags.

Syntax: `<%@ directive attribute="value" %>`

There are three types of directives used in JSP documents: **page**, **include** and **taglib**.

- **The page Directive:** The page directive defines attributes that apply to an entire JSP page, such as the size of the allocated buffer, imported packages and classes/interfaces, and the name of the page that should be used to report run time errors. The page Directive is a JSP element that provides global information about an entire JSP page. This information will directly affect the compilation of the JSP document. The syntax of JSP page directive is as under:

`<%@ page attribute = "value" %>`

An example of the use of page directive is as follows:

```
<%@ page import="java.io.*, java.util.Date" buffer="16k" autoFlush="false" %>
```

```
<%@ page errorPage="error.jsp" %>
```

- **The include Directive:** JSP include directive is used to include files such as HTML, JSP into the current JSP document at the translation time. It means that it enables you to import the content of another static file into a current JSP page. The advantage is code reusability. This directive can appear anywhere in a JSP document. The syntax of include directive is as follows: `<%@ include file = "relative URL" %>`

- **The *taglib* Directive:** This directive allows users to use Custom tags in JSP. A custom tag is user-defined tag. The custom tag eliminates the need for scriptlet tag. It is a reusable code in a JSP page and tag library is a collection of custom tags. The taglib directive has the following syntax:

```
<%@ taglib uri="tagLibraryURI" prefix="tagPrefix" %>
```

2.) Scripting Elements:

Explain JSP scripting elements with the help of a program. 10

JSP scripting elements is a mechanism for embedding java code fragments directly into a HTML page. There are three scripting language elements such as **declarations**, **expressions**, **scriptlets** involved in JSP scripting. Each of these scripting elements has an appropriate location in the generated servlet.

- **Declarations:** Declarations are used to declare the variables and methods that you can use in the JSP document. The declaration part is initialized when the JSP document is initialized. After the declarations have been initialized, they are available to other expressions, declarations and scriptlets. A declaration starts with `<%!` and ends with `%>`. It has the following syntax: `<%! declaration %>`

Following is an example for JSP Declarations:

```
<%! x=0; %> // x is an integer type variable
```

- **Expressions:** The code placed within JSP expression element is written to the output stream of the response. You need not write `out.print()` to write data. Expressions are evaluated at request time, and the result is inserted where the expression appears in the JSP file. The syntax of the expression is as follows: `<%= expression %>`

For example, to show the today date and time: `<%= new java.util.Date() %>`

- **Scriptlets:** A scriptlet element is used to execute java source code in JSP. It is executed at the request time and makes use of declarations, expressions and Java Beans. You can write scriptlets anywhere in a page. It contains a valid java statement within the `<%` and `%>` tag and gets inserted into the `_jspService()` method of generated servlet. The syntax of the scriptlet is as follows:

```
<% // java code %>
```

- 3) **Action Elements:** Action elements or Standard actions tags that can be embedded in a JSP program. JSP provides many standard action tags that you can use for specific tasks such as add java bean objects in JSP document, forward the request to another resource or include other resources in current resource, etc. At the compile time, they are replaced by java code.

JSP Action	Description
<code><jsp:useBean></code>	To get the java bean object from given scope or to create a new object of java bean.
<code><jsp:getProperty></code>	To get the property of a java bean, used with <code>jsp:useBean</code> action.
<code><jsp:setProperty></code>	To set the property of a java bean object, used with <code>jsp:useBean</code> action.
<code><jsp:param></code>	To define the name/value pair of parameter to be passed to <code><jsp:useBean></code> , <code><jsp:forward></code> , <code><jsp:include></code> and <code><jsp:plugin></code> tag.
<code><jsp:forward></code>	To forward the request to another resource.
<code><jsp:include></code>	To include a resource at runtime, the recourse may be HTML, JSP or any other file
<code><jsp:plugin></code>	To embed another component such as applet in JSP

JSP Implicit objects or predefined variables are the java objects that you can use directly without being declared first in scriptlets of JSP document. JSP implicit objects are created during the translation phase of JSP to the servlet. These variables are automatically available for the JSP page developer to use.

1. **out Object:** This implicit object is a simple and frequently used in scriptlet of JSP page. You call either its `print()` or `println()` method to send output to the client browser.

For example, `<% out.println(" JSP is an easy language"); %>`

2. **request Object:** The request object is an instance of `HttpServletRequest` interface. This object is used to retrieve the values that the client browser passed to the server during HTTP request, such as cookies, session, headers information, or parameters associated with the request. The most common use of request object is to access queries by calling the `getParameter()` and `getQueryString()` method.

For example: `<% String name=request.getQueryString(); %>`

3. **response Object :** The response object is an instance of `HttpServletResponse` interface. It is used to add or manipulate response such as redirect response to another resource, send error etc. Using response object, a reply is sent back to the client browser.

`<% response.sendRedirect("http://www.ignou.ac.in");%>`

For example, when the above code is executed, the `sendRedirect()` method of the `javax.servlet.HttpServletResponse` to redirect the user to IGNOU website.

4. **session Object :** The session object is represented by the `javax.servlet.http.HttpSession` interface. This object is most often used when there is a need to share information between requests for a single user. It is used to store the user's data to make it available on other JSP pages until the user session is active. This object is used to track the user information in the same session.
5. **application Object :** The application object is an instance of `javax.servlet.ServletContext`. The `ServletContext` object is created only once by the web container when application or project is deployed on the server. This object is used to get initialization parameter from configuration file. There is one `ServletContext` object for each web application per Java Virtual Machine.
6. **page Object :** The page object is an instance of `javax.servlet.jsp.HttpJspPage` interface. The page object is just as it sounds, a reference to the current instance of the JSP page. The page object is a synonym for 'this' reference and is not useful for programming language.
7. **exception Object :** This implicit object only exists in a defined errorPage. It contains reference to uncaught exception that caused the error page to be invoked. This object is assigned to the `Throwable` class, which is the super class of all errors and exceptions in the java language. You can find a complete description of errorPage mechanism including the use of this exception object in section 4.8 (exception handling in JSP) of this unit.
8. **config Object :** The config object is an instance of `javax.servlet.http.servletConfig`. The container for each jsp page creates it. This object is used to get the configuration information of the particular JSP page by using the `getServletConfig` method. This can be useful in retrieving standard global information such as the paths or file locations.
9. **Exception Object:** This implicit object only exists in a defined errorPage. It contains reference to uncaught exception that caused the error page to be invoked. This object is assigned to the `Throwable` class, which is the super class of all errors and exceptions in the java language

The JavaServer Pages Standard Tag Library (JSTL) is a collection of useful JSP tags which encapsulates the core functionality common to many JSP applications.

JSTL has support for common, structural tasks such as iteration and conditionals, tags for manipulating XML documents, internationalization tags, and SQL tags. It also provides a framework for integrating the existing custom tags with the JSTL tags.

The JSTL tags can be classified, according to their functions, into the following JSTL tag library groups that can be used when creating a JSP page –

Core Tags	Formatting tags	SQL tags	XML tags	JSTL Functions
------------------	------------------------	-----------------	-----------------	-----------------------

- **JSTL Core Tags:** The core group of tags are the most commonly used JSTL tags. To use these tags in JSP, we should have used the following taglib: `<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>`
- **JSTL Formatting Tags:** The Formatting tags provide support for message formatting, number and date formatting. To use these tags in JSP, we should have used the following taglib:
`<%@ taglib prefix = "fmt" uri = "http://java.sun.com/jsp/jstl/fmt" %>`
- **JSTL XML tags:** These tags provide support for creating and manipulating the XML documents. The JSTL XML tag library has custom tags used for interacting with the XML data. This includes parsing the XML, transforming the XML data and the flow control based on the XPath expressions. To use these tags in JSP, you should have used the following taglib:
`<%@ taglib prefix = "x" uri = "http://java.sun.com/jsp/jstl/xml" %>`
- **JSTL Functions:** JSTL includes a number of standard functions; most of them are common string manipulation functions. To use these tags in JSP, you should have used the following taglib:
`<%@ taglib prefix = "fn" uri = "http://java.sun.com/jsp/jstl/functions" %>`
- **JSTL SQL Tags:** The JSTL SQL tag library provides tags to access the database such as Oracle, MySQL, or Microsoft SQL Server. To use these tags in JSP, you should be used the following taglib:
`<%@ taglib prefix = "sql" uri = "http://java.sun.com/jsp/jstl/sql" %>`

There are two important JSTL SQL tag

- `<sql:setDataSource>` : This tag is used to create a DataSource configuration that may be stored into a variable that can be used as an input to another JSTL database-action.
`<sql:setDataSource var="name_of_variable" driver="name_of_driver" url="jdbc_url" user=" " password=" " />`
- JSTL `<sql:query>` tag :This tag is used for executing the SQL query written into its body or through the sql attribute. For example:
`<sql:query dataSource="${db}" var="name_of_variable">`
SELECT ProgCode, Pname from programme;
`</sql:query>`

EXCEPTION HANDLING IN JSP

When writing a JSP code, there is a chance that might make coding errors that can occur in any part of the code. In Java Server Pages (JSP), *there are two types of errors*. The first type of error comes at translation or compilation time when JSP page is translated from JSP source file to Java Servlet class file. The second type of JSP error which is called a request time error occurs during run time or request time. These run time errors result in the form of exception. Exception Handling is the process to handle runtime errors. *There are three ways to perform exception handling in JSP*. They are:

- `errorPage` and `isErrorPage` attributes of page directive
- `<error-page>` tag in Deployment Descriptor file
- try and catch block in Scriptlets

1. `errorPage` and `isErrorPage` attributes of page directive:

- **The *errorPage* attribute** : The `errorPage` attribute of page directive is used to specify the name of error page that handles the exception. The error page contains the exception handling code description for a particular page. The syntax of this attribute is as follows:
`<%@ page errorPage="relative URL" %>`
- **The *isErrorPage* attribute** : The `isErrorPage` attribute of page directive indicates whether or not the JSP page is an errorPage. The default value of this attribute is false.
`<%@ page isErrorPage="true" %>`

2. Using try and catch block in Scriptlets

If you want to handle exception within the current JSP page, you can also use the standard java exception handling mechanism in the Scriptlets component of the JSP page. For this, you can use try and catch clause in your source code. In try block, you can write the normal code that might throw an exception. The catch block is used to handle the exception. Both the try and catch clauses are written inside the scriptlets. Unlike page directive, there is no need to write an error page for this mechanism. You can use the try and catch block like the following manner:

```
<% try {  
    // code that thrown an exception  
} catch (exception e) {  
    // exception handler for exception occur in try block  
} %>
```

3. Using `<error-page>` tag in Deployment Descriptor file

If you want to describe the exception handling in the JSP page at the application level, you can use the `<errorpage>` tag element in the Deployment Descriptor file. The deployment descriptor is a file named `web.xml`. It resides in the web applications under the `WEB-INF/` directory. To include a generic error page for all exceptions at application level in the following way in `web.xml` file:

```
<error-page>  
    <exception-type>Exception</exception-type>  
    <location>/error.jsp</location>  
</error-page>
```

{Block 2}

{CH=5}

*STRUTS 2.x FRAMEWORK

Struts1 was an open-source framework to develop Java EE web applications. Struts1 was created by Craig McClanahan and given to the Apache Foundation. Apache Struts 2.x is a web application framework based on Open Symphony. It supports the complete application development cycle from the build to maintenance phase built on the MVC design pattern. In a typical Java web application, a client calls the server using a web form. Subsequently, the information pass to a Java Servlet that produces an HTML-formatted response after interaction with the database.

Strut2 is an MVC based framework where the application logic is isolated from the user interface layer. The application logic interacts with the database in the Model and is the lowest level of the pattern. The View is responsible for exhibiting data in the form of HTML pages to the client. The Controller holds the instances or the software code that passes the information between the Model and View.

Struts 2 features

- **POJO Forms and POJO Actions:** Struts2 allows using any Plain Old Java Object (POJO) to get the form input or Action class. The POJO prevents the developers from interface implementation or inheritance of any class.
- **Tag Support:** Struts2 allows developers to use new tags where they have to write less code and so they are code efficient form tags
- **AJAX support:** Struts2 supports the asynchronous requests using the AJAX tags. AJAX is beneficial in improving the performance by only sending the required field data and avoiding unnecessary information.
- **Easy integration:** Struts2 supports easy integration with other frameworks like Spring, Tiles, hibernate etc.
- **Easy to modify tags:** Struts2 supports easy tag modification, requiring only basic HTML, XML and CSS knowledge.
- **Promote less configuration:** Struts2 supports default values for the various settings, which provide ease of access.
- **View Technologies:** Struts2 supports multiple view options like JSP, Free marker, Velocity etc.

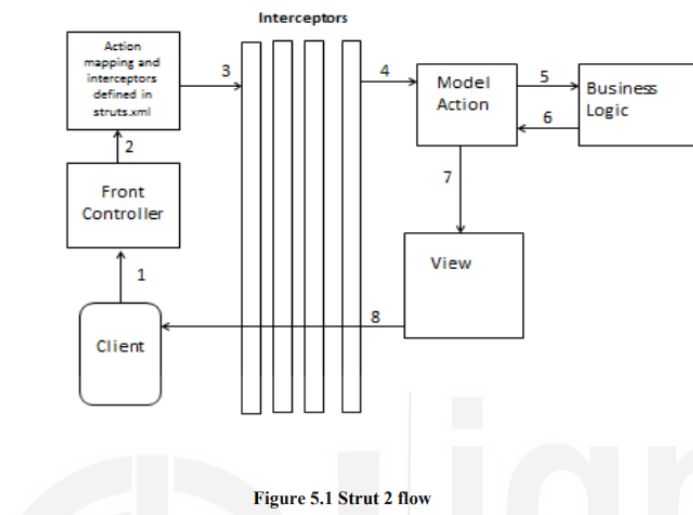
Struts 2 Components

- **Configuration Files:** This component creates the configuration files to couple the Action, View and Controllers like struts.xml, web.xml, struts.properties. There are two main configuration files i.e. struts.xml and struts.properties file, where struts.xml file is used to override the applications default settings and struts.properties file is used to modify the behaviour of the framework. The struts.xml file is created within the WEB_INF/classes directory and holds the configuration information to be modified as actions. The struts.properties file allows us to change the framework properties as per the requirements.
- **Action:** This component contains an action class that controls the user's interaction, the Model, and the View and holds the complete business logic and helps in the data processing. The Action class also responds to a user request and allows the framework to return the result back to the user based on the configuration file
- **Interceptors:** This component is a part of the Controller, and it creates interceptors or uses existing interceptors as required. These are like servlet filters and execute before and after the processing of the request. Generally, they perform common actions like session logging, validation etc., for different actions.
- **View:** This component creates JSPs to control the user's interaction to take input and display the final message.
- **VALUESTACK:** This component is a storage area used to store associated data during processing, i.e Temporary, Model, Action and Named objects.

- **Object Graph Navigation Language (OGNL):** This component provides the functionality of retrieving VALUESTACK data and helps in converting data types.

*Working and flow of Struts 2

1. The client/user sends an HTTP resource request to the server. It reaches to ServletContainer.
2. Web Container loads the web.xml and verifies the URL pattern if it matches; web Container forwards the request to StrutsPrepareAndExecuteFilter (Front Controller)
3. Based on the request URL mapping in struts.xml, the Filter dispatcher identifies the appropriate action class to execute.
4. Before the Action class is executed, the request is passed through the stack of interceptors. Interceptor allows common tasks defined in clean, reusable components such as workflow, validation, file upload etc., that you can keep separate from your action code.
5. The action class calls business logic. Action executed through the action methods performing database operations of storing or retrieving data.
6. Processed data from business logic sent back to the Action class.
7. Based on the result, the Controller identifies the View rendered. Before the response is generated, the stack of interceptors is executed again in the reverse order performing clean-up etc
8. View rendered to the user through the servlet controller.



Explain the working and flow of struts2.
(10)

* SPRING FRAMEWORK:

The spring framework is a lightweight and open-source Java platform that provides solutions to various technical problems through J2EE applications. Spring is assumed as a framework of frameworks that supports other frameworks like struts, hibernate etc., through extensive programming and configuration model. The spring framework is used to develop simple, reliable, and scalable Java applications. It builds web applications on top of the Java EE platform with different extensions.

The fundamental design principle of the Spring framework is "Open for extension, closed for modification". WQ It uses various new techniques to make easier J2EE development and eliminate the complications of developing enterprise applications. Different methods like Dependency Injection, Inversion of Control, Plain Old Java Object (POJO) and Aspect-Oriented Programming (AOP) are used to develop enterprise applications.

Spring framework Advantages/Benefits

- **POJO** - It allows developers to use POJOs for enterprise-class applications development, making them evade using an EJB container like an application server instead of using a servlet container like Tomcat.
- **Modular** - It supports a large number of modules, and due to the modular nature of Spring, the developer considers only the one they require for the development.

- **Integration** - It offers good integration with existing frameworks like ORM, logging frameworks and other view technologies etc.
- **Testability** - It offers simple application testing due to the use of POJOs. The environment-dependent code moves directly into the framework for injecting test data that supports dependency injection.
- **Web MVC** - It offers an elegant web MVC framework that delivers better options than other web frameworks such as Struts etc.
- **Central Exception Handling** – It offers an API that translates technology specific exceptions into consistent, unchecked exceptions.
- **Lightweight** – It is lightweight compared to EJB containers and is very efficient with low resources like memory and CPU to develop and deploy applications.
- **Transaction management** - It provides proper scalability for the consistent transaction management interface. It supports a global transaction to local transactions by scaling up and down as required.

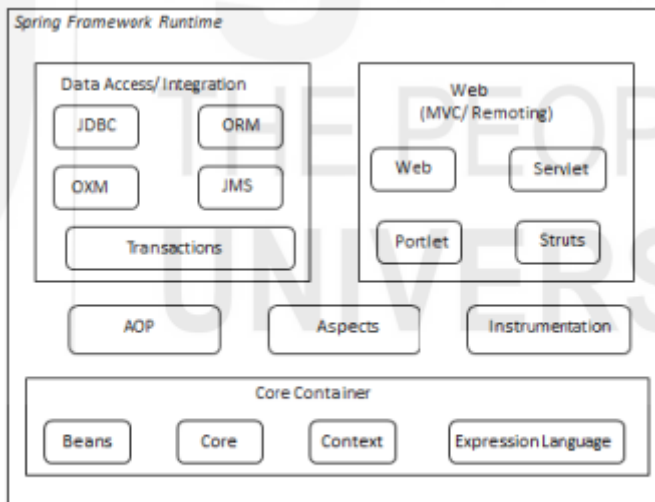


Figure 5.2: Spring Framework

Another important concept of the spring framework is **Aspect-Oriented Programming**

Aspect-oriented programming – It is a programming style where instead of OOPS objects, the aspects are used. An aspect is a class that spans multiple application nodes distributed across methods, classes, object hierarchies and object models, such as logging, security, transaction etc. AOP defines specific policies and offers modularity, and in this paradigm, the program logic is broken down into distinct parts called concerns, enabling the application to adapt to changes.

*Spring MVC Framework

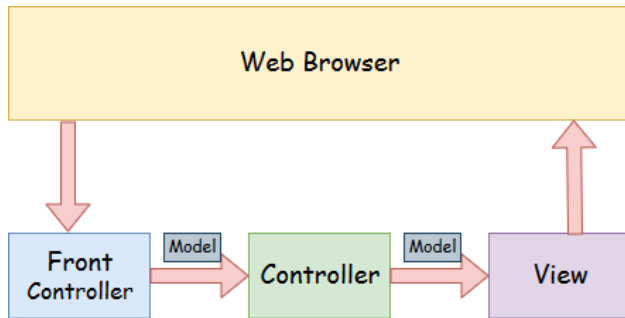
A Spring MVC is a Java framework which is used to build web applications. It follows the Model-View-Controller design pattern. It implements all the basic features of a core spring framework like Inversion of Control, Dependency Injection. A Spring MVC provides an elegant solution to use MVC in spring framework by the help of **DispatcherServlet**. Here, **DispatcherServlet** is a class that receives the incoming request and maps it to the right resource such as controllers, models, and views.

The **dependency injection (DI)** is the core concept of the **Spring framework**, which enables the development of loosely coupled and easily unit testable applications. Based on the DI, Spring framework created various modules such as **Spring MVC, Spring JDBC, Spring Test, Spring ORM, Spring AOP** etc.

Spring Web MVC framework enables us to develop **decoupled web applications**. It introduces the concepts of Dispatcher Servlet, ModelAndView and View Resolvers to develop loosely coupled web applications.

- **Model** - A model contains the data of the application. A data can be a single object or a collection of objects.

- **Controller** - A controller contains the business logic of an application. Here, the @Controller annotation is used to mark the class as the controller.
- **View** - A view represents the provided information in a particular format. Generally, JSP+JSTL is used to create a view page. Although spring also supports other view technologies such as Apache Velocity, Thymeleaf and FreeMarker.
- **Front Controller** - In Spring Web MVC, the DispatcherServlet class works as the front controller. It is responsible to manage the flow of the Spring MVC application.



Spring MVC Configuration Process. (10)

Explain Spring MVC framework with help of a suitable diagram. 10

Spring MVC Configuration Process

To create a simple Spring MVC application, we need to perform the following steps:

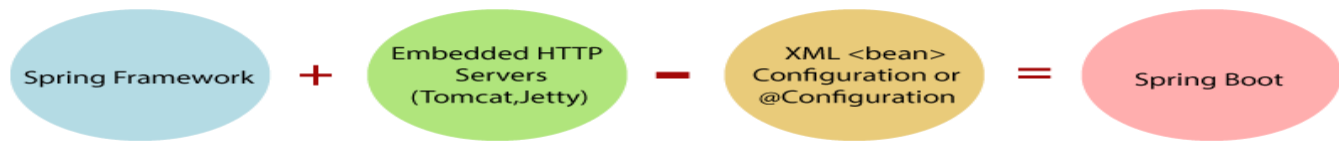
1. We need to add the spring context and spring web MVC dependencies to use the Spring MVC framework in the java project. We add spring-mvc.jar in the application classpath for the java project and add the jar file in the /WEBINF/lib folder for the web application.
2. Configure the DispatcherServlet through the web.xml file for handling requests through the spring container. For the application deployment, the Servlet container creates an instance for the ContextLoaderListner, which facilitates the loading of the webApplicationContext.
3. Configure the spring file to define beans for annotations usage and configure the view resolver for view pages.
4. Configure the web request mappings to handle the client requests.

Spring MVC Framework advantages

- Separate roles - The different components of the flow are parts of the applicationContext that extends the plainApplicationContext. A specialized object fulfils the position of each element.
- Lightweight – To develop and deploy an application, the Spring MVC Framework uses a lightweight servlet container.
- Robust Configuration - Spring MVC offers a strong configuration for the framework and application classes.
- Reusable business code – Spring MVC Framework offers reusable code that permits usage of existing objects instead of creating a new one.

*Spring Boot

Spring Boot is a project that is built on the top of the Spring Framework. It provides an easier and faster way to set up, configure, and run both simple and web-based applications. It is a Spring module that provides the **RAD (Rapid Application Development)** feature to the Spring Framework. It is used to create a stand-alone Spring-based application that you can just run because it needs minimal Spring configuration.



In short, Spring Boot is the combination of **Spring Framework** and **Embedded Servers**. In Spring Boot, there is no requirement for XML configuration (deployment descriptor). It uses convention over configuration software design paradigm that means it decreases the effort of the developer.

Why should we use Spring Boot Framework?

We should use Spring Boot Framework because:

- The dependency injection approach is used in Spring Boot.
- It contains powerful database transaction management capabilities.
- It simplifies integration with other Java frameworks like JPA/Hibernate ORM, Struts, etc.
- It reduces the cost and development time of the application.

Advantages of Spring Boot

- It creates **stand-alone** Spring applications that can be started using Java **-jar**.
- It tests web applications easily with the help of different **Embedded** HTTP servers such as **Tomcat, Jetty**, etc. We don't need to deploy WAR files.
- It provides opinionated '**starter**' POMs to simplify our Maven configuration.
- There is no requirement for **XML** configuration.
- It offers a **CLI** tool for developing and testing the Spring Boot application.
- It offers the number of **plug-ins**.
- It **increases productivity** and reduces development time.

Spring Boot components :The essential Spring Boot components are:

- 1) Spring Boot Starter 2) Spring Boot autoconfiguration 3) Sprng Initializer 4) Sprg Boot Actuator 5) Spring Boot CLI

Struts Vs Spring

<u>Struts</u>	<u>Spring</u>
An open source framework which enables to extend Java Servlet API and MVC framework.	An open source framework to implement inversion of Control (IOC) and dependency Injection (DI).
It is a Heavyweight framework.	It is a Lightweight framework.
It is less flexible than Spring.	It is more flexible than Struts.
It has non layered architecture.	It has layered architecture.
It is tightly coupled.	It is loosely coupled.
It also provides integration with ORM and JDBC but manual coding is required.	It provides an easy integration with ORM and JDBC technologies.

Spring Boot Vs Spring MVC

<u>Spring Boot</u>	<u>Spring MVC</u>
Spring Boot removes all boilerplate code and supports the auto configuration based on jars available into classpath.	Spring MVC requires a lot of configuration and it contains a lot of boilerplate code.
Spring Boot provides many spring boot starters. Spring boot starter is a unit of related dependencies wrapped together. It solves the problem of dependency management.	Dependency management is a tough process since every dependency with the correct version needs to be declared.
Spring Boot makes the application development easier and faster.	Spring MVC requires a lot of configurations and dependency management is time consuming. Hence, development is slow.
Spring Boot supports embedded servers to be packaged along with applications to run the jar stand alone.	A lot of manual configuration is required to attain the same level of packaging.
Spring Boot supports many production ready features such as Actuators, Process monitoring out-of-box.	Spring MVC does not support any production ready feature.
Spring Boot is very flexible and provides many modules which can be used to build many different other applications.	Spring MVC is designed only for the development of dynamic web pages and RESTful web services.

Spring Vs Hibernate

<u>Spring</u>	<u>Hibernate</u>
Spring is an open source, complete and modular application framework to develop enterprise applications in java.	Hibernate is an ORM framework specialized in data persisting and data retrieval from DB.
Spring provides many useful features such as transaction management, aspect-oriented programming and dependency injection (DI).	Hibernate provides Object-Relational Persistence and Query service for applications.
Spring framework has many modules such as Spring-Core, Spring-MVC, Spring-Rest, Spring-Security and many more.	Hibernate does not have modules like Spring framework.
Spring framework has support for connection pooling by changing the configuration in the spring configuration file.	Hibernate supports robust connection pooling features. Hibernate supports two levels of cache which improves the application performance.

Java Annotations

Java **Annotation** is a tag that represents the *metadata* i.e. attached with class, interface, methods or fields to indicate some additional information which can be used by java compiler and JVM.

Annotations in Java are used to provide additional information, so it is an alternative option for XML and Java marker interfaces.

Built-In Java Annotations used in Java code

- @Override
- @SuppressWarnings
- @Deprecated

@Override

@Override annotation assures that the subclass method is overriding the parent class method. If it is not so, compile time error occurs.

Sometimes, we do the silly mistake such as spelling mistakes etc. So, it is better to mark @Override annotation that provides assurance that method is overridden.

```
class Animal {
void eatSomething() {
System.out.println("eating something"); }
}
class Dog extends Animal {
@Override
void eatsomething() {
System.out.println("eating foods"); } //should be eatSomething
}
```

```
class TestAnnotation1 {
public static void main(String args[]) {
Animal a=new Dog();
a.eatSomething();
}}
```

@SuppressWarnings

@SuppressWarnings annotation: is used to suppress warnings issued by the compiler.

```
import java.util.*;
class TestAnnotation2{
@SuppressWarnings("unchecked")
public static void main(String args[]){
ArrayList list=new ArrayList();
list.add("sonoo");
list.add("vimal");
list.add("ratan");
}
```

```
for(Object obj:list)
System.out.println(obj);
}}
```

@Deprecated

@Deprecated annotation marks that this method is deprecated so compiler prints warning. It informs user that it may be removed in the future versions. So, it is better not to use such methods.

```
class A{
void m() { System.out.println("hello m"); }

@Deprecated
void n() { System.out.println("hello n"); } }

class TestAnnotation3 {
public static void main(String args[]) {
A a=new A();
a.n();
}}
```

Built-in Annotations used in custom annotations in java

- @Target
- @Retention
- @Inherited
- @Documented

@Target: @Target tag is used to specify at which type, the annotation is used.

```
@Target(ElementType.TYPE)
@interface MyAnnotation{
int value1();
String value2();
}
```

@Retention: @Retention annotation is used to specify to what level annotation will be available.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface MyAnnotation{
int value1();
String value2();
}
```

@Inherited: allows inheritance where we can apply an annotation to other annotation. Annotation inheritance is not available by default and so not available to child class from the parent class.

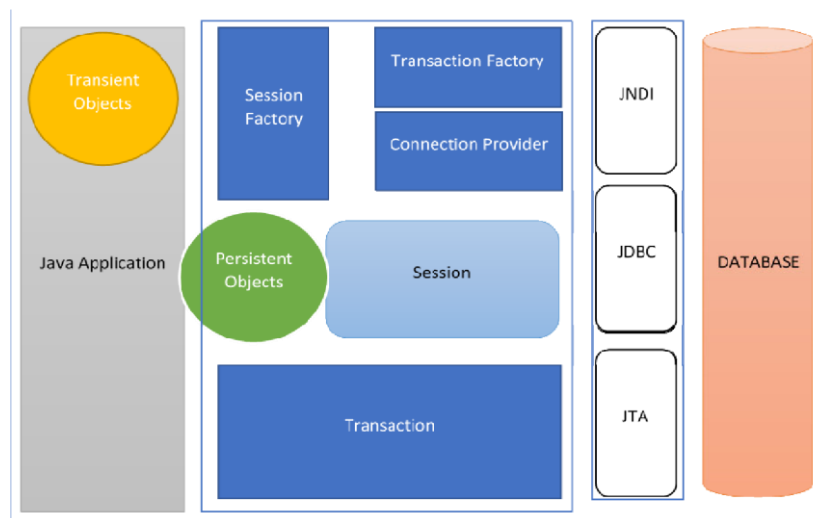
@Repeatable: annotation facilitates annotation more than once as, by default, it is applied once on a java element.

*Hibernate Architecture

The Hibernate framework is built on top of existing Java API, and the architecture is characterized into four layers viz. Database, Back-end API, Hibernate framework, and Java application layer.

The core components of Hibernate architecture are:

- Configuration object holds the configuration properties viz. database configuration file and class mapping files.
- SessionFactory is a factory of sessions that provides a factory method to get the session object. It is a heavyweight object, immutable and is available to all the sessions. It is created at the time of application startup for later use and persists till the Hibernate is running.
- Session objects provide an interface between the application and the database. It is a lightweight object and gets instantiated whenever an application requires a database interaction. The session object offers a method to create, read, update and delete operations.
- TransactionFactory is a factory of Transaction that offers a method for transaction management.
- ConnectionProvider is an optional factory of JDBC connections.
- Transient or Persistent objects are database objects. In the transient states the new objects created in the Java program are not associated with any hibernate session. Still, in the case of the persistent state the object is associated with a hibernate session.



{CH=6}

Model 1 Architecture

In model 1 architecture, JSP pages were the focal point for the entire web application. JSP provides the solutions to the problems of Servlet technology. JSP provides a better separation of concern not to mix business logic and presentation logic. Re Deployment of web applications is not required if the JSP page is modified. JSP supports JavaBean, custom tags and JSTL to keep the Business logic separate from JSP. Model 1 architectural diagram is shown below.

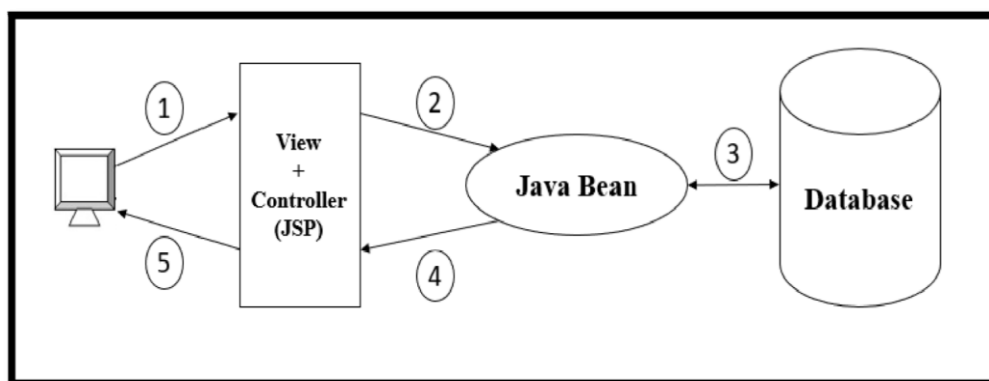


Figure 6.1 : Model 1 Architecture

Notice that there is no **Servlet** involved in the process. The client request is sent directly to a JSP page, which may communicate with JavaBeans or other services, but ultimately the JSP page selects the next page for the client.

Advantages

- Quick and easy way to develop a web application

Disadvantages

- **Decentralized navigation Control:** There is no central control to determine the navigation since every JSP page contains the logic to determine the next page.
 - **Hard to maintain:** If a JSP is renamed, then every other JSP which refers to the renamed JSP, needs to be modified.
 - **Time Consuming:** Reusable components like custom tags development requires more time.
 - **Difficult to extend:** Model1 architecture is not suitable for large and complex applications.
-

Model 2 (MVC) Architecture

In Model 2 architecture contrast to the Model 1 architecture, a Servlet known as **Controller Servlet** intercepts all client requests. The Controller Servlet performs all the initial request processing and determines which JSP to display next.

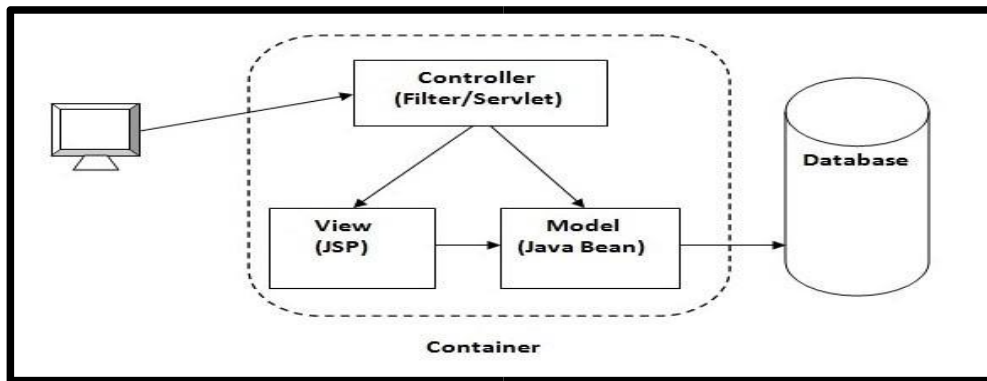


Figure 6.1: Model 2 (MVC) Architecture

In this architecture, the client never sends requests directly to the JSP. This architecture enables us to perform authentication and authorization, centralized logging etc. Once request processing is finished, the servlet directs the request to the appropriate JSP page.

As it can be observed that the key difference between the two architectures is the **Controller Servlet** introduced into Model 2, which provides a single point of entry and encourages more reuse and extensibility than the Model 1 architecture. The Model 2 architecture clearly provides the separation of business logic, presentation logic, and request processing. This separation is commonly known as **Model-View-Controller (MVC)** pattern.

Advantages

- Centralized navigation Control
- Easy to maintain
- Separation of concern
- Easy to extend
- Easy to test

Disadvantages

- Developer writes the Controller Servlet code and it needs to be compiled and redeployed if any logic changes for Controller Servlet.
-

*MAVEN

Maven has a built-in powerful dependency management feature. To make the project dependencies available into classpath, we just need to specify all the required dependencies with GAV (groupId, artifactId, version) as shown in sample pom.xml. Maven downloads all the dependencies recursively and puts them into the local **maven repository**.

- **Maven Repositories:** Maven repositories are directories of packaged JAR files with extra Meta data. There are three types of repository in Maven.
 - **Local Repository** – As the name indicates, it's a repository located on the developer's machine itself. Maven downloads the dependencies from Central repository, Remote repositories and stores it into the Local Repository. By default, the Local Repository location is **user-home/.m2**.
 - **Central Repository** – The Central Repository is maintained and provided by the maven community. The Central Repository access does not require any specific configuration.
 - **Remote Repository** – A Remote Repository is like the Central Repository from where maven can download the dependencies and store it into Local Repository. It may be located on any web server on the internet or the local network. It can be configured into pom.xml by putting the following contents just after **<dependencies>** element.

Maven follows a **build life cycle** while building an application. The build life cycle consists of phases and phase consists of build goals.

- **Build Life Cycle:** Maven has 3 built-in build life cycles which are responsible for different aspects of building a software project. These build life cycles execute independent of one another. For two different build life cycles, you have to use two different maven commands and these build life cycles will execute sequentially. The build life cycles are as followings-
 - **default** - The *default* build lifecycle is of most interest since this is what builds the code. It is responsible to handle everything related to compiling and packaging the software project. ***This build life cycle can't be executed directly.*** You need to execute a build **phase** or **goal** from the default build life cycle.
 - **clean** - The *clean* build life cycle performs the tasks related to cleaning such as deleting compiled classes, removing previous jar files, deleting the generated source codes, removing temporary files from the output directory etc. The command **mvn clean** cleans the project.
 - **site** - The *site* build life cycle performs the task related to generating documents for the project.
- **Build Phases:** A Maven phase represents a stage in the Maven build life cycle. Each phase performs a specific task. The most commonly used build phases defined for default build life cycle are as follows.

Build Phase	Description
validate	Verifies the correctness of the project and make sure that all required dependencies are downloaded.
Compile	Compiles the source code to produce the binary artifacts

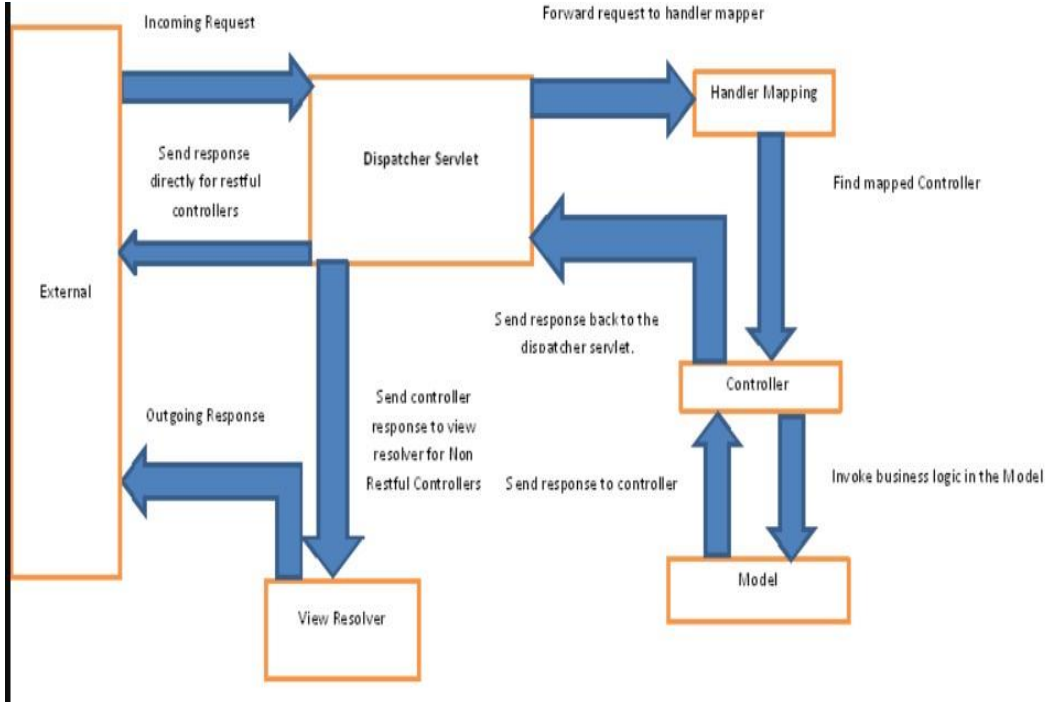
Test	Executes the unit test cases using a suitable unit testing framework without packaging the binary artifacts
Package	Packs the binary artifacts into distributable format such as JAR, WAR, EAR
intergrationtest	Executes integration test cases which require packaging.

- Build Goals:** The finest step in the maven build process is a Goal. A goal can be bound to zero or more build phases. A phase helps to determine the order in which the *goals* are executed.

{CH=7}: Practical book me se cover krna h..isse upr vle chapters me bhi h

CREATING CONTROLLERS AND VIEWS :In Spring MVC, controller classes are used to handle the request which are coming from the client. After getting the request from client then controller starts its task by invoking a business class to process business-related tasks, and once it gets fulfilled then it redirects the client request to a logical view name, and now it is resolved by Spring’s dispatcher servlet in order to render results or output. Below are the main responsibilities:

- Controllers main task is to intercept the incoming requests.
- Controller Converts the payload of the request to the internal structure of the data (mapping basically).
- After getting response from above two steps then it sends the data to Model for further processing,
- Once it gets the processed data from the Model then that data is referred to the View for rendering/display.



High Level Flow in Spring MVC

practical h baki sara book me se cover krna h

CONFIGURATION OF BOOTSTRAP IN SPRING APPLICATION

Bootstrap was originally created by a designer and developer at Twitter in mid 2010. Prior to being an open-sourced framework, Bootstrap was known as *Twitter Blueprint*. **Bootstrap** is a free, popular and open-source CSS framework directed at responsive, mobile-first front-end web development. Unlike many web frameworks, it concerns itself with front-end development only. It contains HTML and CSS based design templates for various interface components such as typography, forms, buttons, modals, image carousels, navigation etc., as well as optional JavaScript extensions. Bootstrap provides the ability to create responsive designs faster and easily.

various factors which support learning Bootstrap are as follows.

- **Browser Support:** Bootstrap supports all popular web browsers such as Chrome, Safari, Firefox, Opera, and Internet edge.
- **Responsive Design:** Bootstrap enables us to write responsive websites very easily. Its responsive CSS adjusts automatically to Desktops, Tablets and Mobiles.
- **Uniform solution to build interface:** Bootstrap provides a clean and uniform solution to build an interface for the developers.
- **Customization:** Bootstrap provides a simple and easy way to customize the CSS.
- **Functional built-in components:** Bootstrap contains beautiful and functional built-in components which can be customized very easily.

Different ways to configure Bootstrap

The Bootstrap 5 is the most recent version of the Bootstrap framework. There are two ways to include Bootstrap into JSP or HTML. The required configurations for Bootstrap 4 and Bootstrap 5 has been described as follows.

1. Bootstrap through Content Delivery Network (CDN)

Bootstrap CDN is an efficient and faster way to deliver the content from your website to the users. Bootstrap CDN speeds up the websites as CDN servers are intelligent enough to serve the resources based on proximity.

BAKI PRACTICAL HAI

SETTING UP DATABASE USING HIBERNATE(Database configuration using Hibernate) Explain process of setting up database using Hibernate. 10

Spring supports bootstrapping the Hibernate SessionFactory in order to set up the database using Hibernate. The database setup can be done with a few lines of Java code or XML configuration. Spring provides two key beans to support the Hibernate integration, available in the org.springframework.orm.hibernate5 package:

- **LocalSessionFactoryBean:** creates a Hibernate's **SessionFactory** which is injected into Hibernate-based DAO classes.
- **PlatformTransactionManager:** provides transaction support code for a **SessionFactory**. Programmers can use **@Transactional** annotation in DAO methods to avoid writing boiler-plate transaction code explicitly.

Practical kafi h

{BLOCK 3}

{ch=9}

Spring Boot Working and Annotations: A class with the main method and annotated with **@SpringBootApplication** is the entry point of the Spring Boot application. Spring Boot auto-configures all required configurations. It performs auto-configuration by scanning the classes in class-path annotated with **@Component** or **@Configuration**. **@SpringBootApplication** annotation comprising the following three annotations with their default values-

- **@EnableAutoConfiguration**
- **@ComponentScan**
- **@Configuration**

Spring Boot Starter: Spring Boot starters are the dependency descriptor that addresses the problem of compatible versions of dependency management. Spring Boot provides a number of starters that make development easier and rapid. Spring Boot starters follow a similar naming pattern as **spring-boot-starter-***, where ***** denotes a particular type of application. Spring Boot provides two runner interfaces named as **ApplicationRunner** and **CommandLineRunner**. These interfaces enable you to execute a piece of code just after the Spring Boot application is started.

Spring Boot DevTools: What is Spring Boot DevTools ? Briefly explain its features. 10

Spring Boot DevTools was released with Spring Boot 1.3. DevTools stands for developer tool. The aim of DevTools module is to enhance the application development experience by reducing the development time of the Spring the code many times and then restarts the application to verify the changed code. DevTools reduces the developer effort. It detects the code changes and restarts the application automatically. DevTools can be integrated into a Spring Boot application by adding dependency into **pom.xml**. Spring Boot DevTools provides the following important features –

- Automatic Restart
- Live Reload
- Property defaults

1. **Automatic Restart:** Auto-restart refers to the reloading of Java classes and configures it at the server-side. While developing an application, a developer changes the code frequently and to verify these changes, the steps build, deploy and restart of the server is required. Spring Boot DevTools automates the build and deploys process, which increases productivity. Files change in the class-path triggers Spring Boot DevTools to restart the application. This auto-restart process reduces the time significantly to verify the changes. Spring Boot uses two types of ClassLoaders:
 - **Base ClassLoader:** This ClassLoader loads the classes which do not change. E.g. Third-party jars
 - **Restart ClassLoader:** This ClassLoader loads the classes which we are actively developing
2. **Live Reload:** Live Reload or auto-refresh is also a very important feature provided by Spring Boot DevTools. Spring Boot DevTools module also comes with an embedded LiveReload server that can be used to trigger a browser refresh whenever a resource is changed. For example, when a developer makes the changes into templates or other resources, he/she has to refresh the browser to verify the changes. With the Live Reload/Auto Refresh feature, developers do not need to press F5 to refresh the browser. Thus, it enhances the development experience and increases productivity.
3. **Property Defaults:** Spring Boot does a lot of auto configuration. It also includes caching for performance improvement. Template technology Thymeleaf contains the property **spring.thymeleaf.cache**. DevTools disables the caching and allows us to update pages without restarting the application. During the

development, caching for Thymeleaf, Freemarker, Groovy Templates are automatically disabled by DevTools.

Spring Boot Actuator

Spring Boot Actuator is a sub-project of the Spring Boot framework. The Actuator provides production-ready features such as application monitoring, Network traffic, State of database and many more. Without any implementation, Actuator provides production-grade tools. In Spring Boot application, Actuator is primarily used to expose operational information about the running application such as info, health, dump, metrics, env etc. It provides **HTTP** and **JMX** endpoints to manage and monitor the Spring Boot application. There are three main features of Spring Boot Actuator-

- Endpoints
- Metrics
- Audit

Endpoints: The actuator endpoints enable us to monitor and interact with the application. In Spring Boot 2.x, most of the endpoints of the Actuator are disabled. By default, only two endpoints **/health** and **/info** are available. Other required endpoints can be enabled by adding **management.endpoints.web.exposure.include** property into application.properties. **By default, all Actuator endpoints are now placed under the /actuator path.** Some of the important endpoints are listed below –

- **/health** provides the health status of the application
- **/info** provides general information about the application. It might be build information or the latest commit information.
- **/metrics** provides metrics of application. Returned metrics include generic metrics as well as a custom metric.
- **/env** provides the current environment properties.

Metrics: Spring Boot integrates the micrometer to provide dimensional metrics. The micrometer is the instrumentation library that empowers the delivery of application metrics from Spring. Metrics of the application can be accessed by **/metrics** endpoint.

Audit: Spring Boot provides a flexible audit framework that publishes events to an AuditEventRepository. It automatically publishes the authentication events if spring-security is in execution.

SPRING BOOT- APPLICATION PROPERTIES

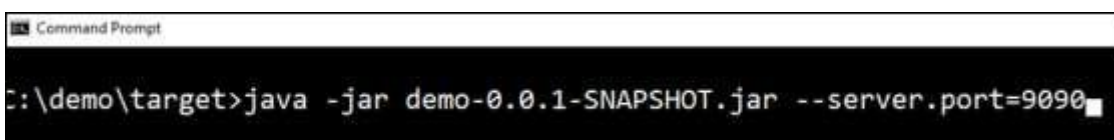
What is Spring Boot? Briefly explain Spring Boot application properties. 10

Application Properties enable us to work in different environments such as Prod, Dev, Test. Spring Boot application properties can be set into properties files, YAML files, command-line arguments and environment variables.

1. **Command Line Properties:** Spring Boot application converts the command line properties into Spring Boot Environment properties. Command line properties take precedence over the other property sources. By default, Spring Boot uses the 8080 port number to start the Tomcat. Let us learn how change the port number by using command line properties.

Step 1 – After creating an executable JAR file, run it by using the command **java -jar <JARFILE>**.

Step 2 – Use the command given in the screenshot given below to change the port number for Spring Boot application by using command line properties.



```
Command Prompt
C:\demo\target>java -jar demo-0.0.1-SNAPSHOT.jar --server.port=9090
```

Note – You can provide more than one application properties by using the delimiter –.

Note: Double hyphen (--) can be used as a delimiter to pass multiple command-line properties.

2. **Properties File:** Properties files are used to keep 'N' number of properties in a single file to run the application in a different environment. In Spring Boot, properties are kept in the **application.properties** file under the classpath.

The application.properties file is located in the **src/main/resources** directory. The code for sample **application.properties** file is given below –

```
server.port = 9090
```

```
spring.application.name = demoservice
```

Property server.port changes the port number on which the web application runs.

3. YAML File

Spring Boot also supports YAML based properties configuration. Properties file application.yml can be used instead of application.properties. **YAML is a convenient format for specifying hierarchical configuration data.** The application.yml file also should be kept in **src/main/resources** directory. Sample application.yml file is shown below-

```
server: port: 9090 spring: application:
```

```
name: demoservice
```

4. Externalized Properties

Spring Boot supports keeping properties file at any location. We can externalize the properties file so that if any property is changed, a new build is not required. Just application restart will take effect on the changed property. While executing the application jar file, the following command-line argument is used to specify the location of the property file.

{CH=10}

Hibernate Architecture

Hibernate has four-layered architecture. The Hibernate includes many objects such as persistent objects, session factory, session, connection factory, transaction factory, transaction etc.

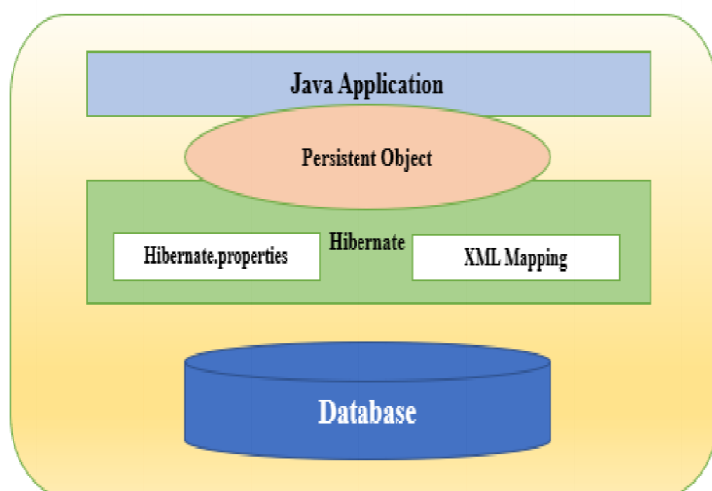


Figure 10.2 Hibernate Architecture

A detailed view of the Hibernate Application Architecture with some of the core classes is shown in Figure 10.3

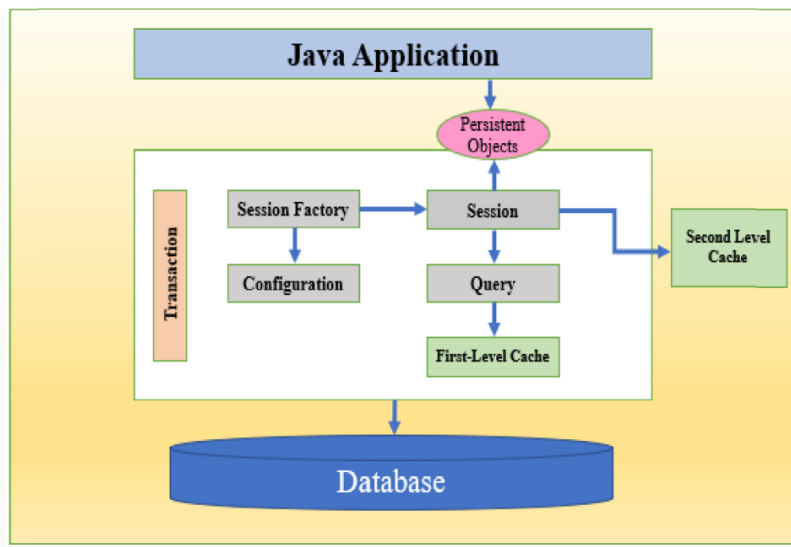


Figure 10.3 Hibernate Detailed Architectural View

Hibernate uses Java APIs like Java Transaction API (JTA), Java Database Connectivity (JDBC) and Java Naming and Directory Interface (JNDI). The knowledge of Hibernate architecture elements helps you to understand the internal working of Hibernate.

Hibernate has a layered architecture which helps the user to operate without having to know the underlying APIs. Hibernate makes use of the database and configuration data to provide persistence services (and persistent objects) to the application.

SessionFactory: The Hibernate creates an immutable SessionFactory object using the Configuration object. SessionFactory is used to instantiate the session object in a thread to service client requests. SessionFactory object is thread safe and used by all the threads of an application. SessionFactory object is per database using a separate configuration file. Thus, for multiple databases, we will require to create multiple SessionFactory objects

Session: The session object provides an interface to interact with the database from an application. It is lightweight, which instantiates each time an interaction is required with the database. Session objects are used to retrieve and save persistent objects. It is a short-lived object which wraps the JDBC connection. It also provides a first-level cache of data.

Transaction: Transaction represents a unit of work with the database, and most of the RDBMS supports transaction functionality. Transaction object provides a method for transaction management. It enables data consistency and rollback in case something wrong.

Query: Query objects use SQL or Hibernate Query Language (HQL) string to retrieve data from the database and create objects. A Query instance is used to bind query parameters, limit the number of results returned by the query, and finally to execute the query.

Persistent Objects: These are plain old java objects (POJOs), which get persisted into the database by hibernate. Persistent objects can be configured in configurations files (hibernate.cfg.xml or hibernate.properties) or annotated with @Entity annotation.

First-level Cache: Cache is a mechanism that improves the performance of any application. Hibernate provides a caching facility also. First-level cache is enabled by default, and it can't be disabled. Hibernate ORM reduces the number of queries made to a database in a single transaction using First-level cache

Second-level Cache: Second-level cache is used globally in Session Factory scope. Once the session factory is closed, all cache associated with it dies, and the cache manager is also closed down. Second level cache validates itself for modified entities, if the modification has been done through hibernate session APIs.

HIBERNATE CONFIGURATION WITH ANNOTATION

Explain Hibernate configuration with annotations. 10

@Entity: In JPA, POJOs are entities that represent the data that can be persisted into a database. An entity represents the **table** in a database. Every instance of an entity represents a **row** in the table. The annotation must be defined at **class level**. The Entity must have no-arg constructor and a primary key. The entity name defaults to the name of the class. Its name can be changed using the name element into @Entity.

```
@Entity(name="student") public class Student
```

```
{  
}
```

@Id: The primary key uniquely identifies the instances of the entity. The primary key is a must for each JPA entity. The **@Id** annotation is used to define the primary key in a JPA entity. Identifiers are generated using

@GeneratedValue annotation. There are four strategies to generate id. The **strategy** element in **@GeneratedValue** can have value from any of **AUTO, TABLE, SEQUENCE, or IDENTITY**.

```
@Entity public class Student
```

```
{
```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
@Column(name = "id") private int id;
```

```
}
```

@Table: By default, the name of the table in the database will be the same as the entity name. **@Table** annotation is used to define the name of the table in the database.

```
@Table(name="STUDENT") public class Student
```

```
{
```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
@Column(name = "id") private int id;
```

```
}
```

```
@Column
```

```
@Table(name="STUDENT") public class Student
```

```
{
```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
@Column(name = "id") private int id;
```

```
}
```

```
public class Student
```

```
{
```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
@Column(name = "id") private int id;
```

```
}
```

```
@Table(name="STUDENT")
```

```
public class Student
```

```
{
```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
@Column(name = "id") private int id;
```

```
}
```

```

@Column(name = "id")
private int id;

@Column(name = "firstname", length=50, nullable=false, unique=false) private String firstName;

@Column(name = "lastname", length=50, nullable=true, unique=false) private String lastName;

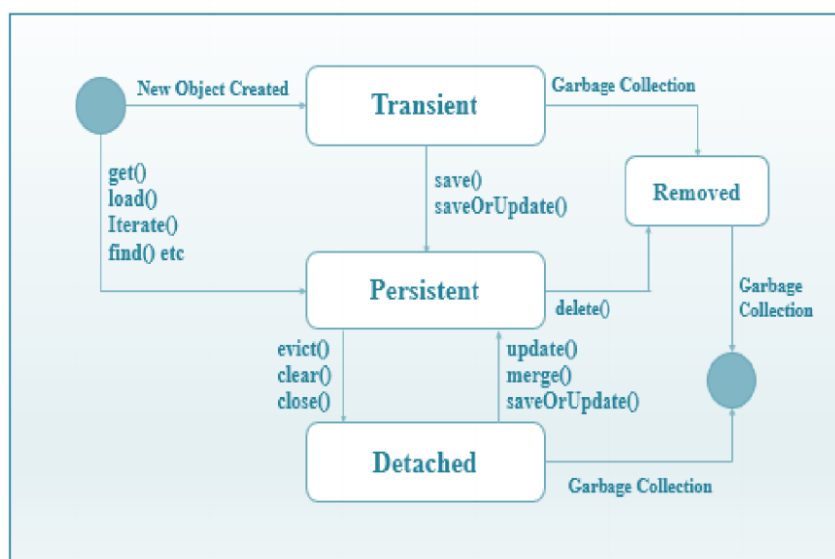
@Transient private int age;

}

```

@Temporal: annotation is used to specify the desired SQL data type. It has a single element *TemporalType*. *TemporalType* can be **DATE**, **TIME** or **TIMESTAMP**. The type of `java.util.Calendar` also requires **@Temporal** annotation to map with the corresponding SQL data type.

Hibernate Entity Lifecycle States



1. **Transient**: An object of a POJO class is in a transient state when created using a new Hibernate (ORM) operator in Java. In the transient state, the object is not related to any database table. An object in a transient state is not managed by session or Entity Manager. The transient state object is not tracked for any modification from the persistence context. Thus, any modification of a transient state object does not impact the database.
2. **Persistent**: When a Transient entity is saved, it is associated with a unique session object and it enters into a Persistent state. A Persistent state entity is a representation of a row into a database. Session manages the Persistent state objects and keeps track of every modification done. Session propagates all the modifications into the database automatically.
3. **Detached**: On call of any session method such as `clear()`, `evict()` or `close()` make the persistent state objects into the detached state. Session does not keep track of any modification to detached state objects. Detached entity has a corresponding row in the database but changes to the entity will not be reflected in the database.
4. **Removed**: Once a persistent object passes through session's `delete()` method, it enters into Removed state. When an application commits the session the entries in the database that correspond to remove entities are deleted. At this state, java instance exists but any change made to the instance will not be saved into the database. If the removed entities are not referenced anywhere, then it is eligible for garbage collection

Hibernate Get Entity – get vs load

Explain the use of load() and get() methods of Hibernate session, with the help of an example. 10

Session load() method: There are several overloaded methods of load() into the Hibernate Session interface. Each load() method requires the object's primary key as an identifier to load the object from the database. Along with the identifier, hibernate also requires to know which class or entity name to use to find the object.

- public Object load(Class<T>theClass, Serializable id) throws HibernateException
- public Object load(String entityName, Serializable id) throws HibernateException
- public void load(Object object, Serializable id) throws HibernateException

Session get() method: The get() method is similar to the load() method to fetch the entity from the database. Like load() method, get() methods also take an identifier and either an entity name or a class. Some important overloaded methods are as follows:

- public Object get(Class<T>clazz, Serializable id) throws HibernateException
- public Object get(String entityName, Serializable id) throws HibernateException

Difference between load() and get() methods

Although load() and get() methods perform the same task, still the differences exist in their return values in case the identifier does not exist in the database.

- The get() method returns NULL if the identifier does not exist.
- The load() method throws a runtime exception if the identifier does not exist.

Hibernate Insert Entity – persist, save and saveOrUpdate

The Session interface into Hibernate provides a couple of methods to transit an object from transient state or detached state to persistent state e.g. persist(), save(), and saveOrUpdate(). These methods are used to store an object in a database.

Session persist() method: The ***persist()*** method is used to add a record into the database. This method adds a new entity instance to persistent context. On call of ***persist()*** method, the transit state object moves into a persistent state object but not yet saved to the database. The insert statement generates only upon committing the transaction, flushing or closing the session. This method has a return type as **void**.

Session save() method: The ***save()*** method is similar to the persist() method which stores the record into the database. This is the original Hibernate Session method which does not conform to JPA specification. The ***save()*** method differs from the persist() method in terms of its implementation. The documentation of this method states that at first, it assigns the generated id to the identifier and then persists the entity. The ***save()*** method returns the **Serializable** value of this identifier.

Session saveOrUpdate() method: This method either performs **save()** or **update()** on the basis of identifier existence. E.g. If an identifier exists for the instance, update() method will be called otherwise save() will be performed. The saveOrUpdate() method handles the cases where we need to save a detached instance. Unlike the save() operation on detached instances, the saveOrUpdate() method does not result in a duplicate record. Similar to update(), this method is used to reattach an instance to the session.

Hibernate Query Language (HQL)

Hibernate Query Language (HQL) is an object-oriented query language. HQL is similar to the database SQL language. The main difference between HQL and SQL is HQL uses class name instead of table name, and property names instead of column name.

HQL Select query to retrieve the student data of id 1.

```
Query query = session.createQuery("from Student where id=:id");
Query.setParameter("id", "1");
List list=query.list();
```

HQL Update query to update the student's last name as 'Singh' whose id 1.

```
Query query = session.createQuery("Update Student set lastName = :lastName where id = :id ");
query.setParameter("lastName", "Singh"); query.setParameter("id", "1"); int result = query.executeUpdate();
```

HQL Delete query to delete the student where id is 1.

```
Query query = session.createQuery("Delete Student where id = :id "); query.setParameter("id", "1"); int result =
query.executeUpdate();
```

HQL Insert In HQL, only the INSERT INTO ... SELECT ... is supported; there is no INSERT INTO ... VALUES. HQL only support insert from another table. For example, insert student records from another student_backup table. This is also called bulk-insert statement.

```
Query query = session.createQuery("Insert into Student(firstName, lastName, emailId)" + "Select firstName,
lastName, emailed from student_backup"); int result = query.executeUpdate();
```

{CH=11}

SPRING DATA REPOSITORY

The **Spring Data repository** aims to reduce the boilerplate code required to implement the DAO layer into a project for various persistence stores. Spring Data repository has different interfaces, each having different functionality. The following interfaces in the Spring Data repository have been explained.

- CrudRepository
- PagingAndSortingRepository
- JpaRepository

Briefly explain Crud Repository of Spring Boot. 10

1. Spring Boot CrudRepository: Spring Boot provides an interface called **CrudRepository** that contains methods for CRUD operations. It is defined in the package **org.springframework.data.repository**. It extends the Spring Data **Repository** interface. It provides generic Crud operation on a repository. If we want to use CrudRepository in an application, we have to create an interface and extend the **CrudRepository**.

Syntax:

public interface CrudRepository<T,ID> **extends** Repository<T,ID>

where, **T** is the domain type that repository manages. **ID** is the type of the id of the entity that repository manages.

For example:

public interface StudentRepository **extends** CrudRepository<Student, Integer>

```
{
}
```

In the above example, we have created an interface named **StudentRepository** that extends **CrudRepository**. Where **Student** is the repository to manage, and **Integer** is the type of Id that is defined in the Student repository.

2. PagingAndSortingRepository : PagingAndSortingRepository interface extends CrudRepository interface. It provides the paging and sorting feature apart from CRUD feature.

Syntax:

public interface PagingAndSortingRepository<T, ID> **extends** CrudRepository<T, ID>

```
{
}
```

3. JpaRepository : JpaRepository extends PagingAndSortingRepository interface, which extends CrudRepository. JpaRepository contains all API of CrudRepository and PagingAndSortingRepository. JpaRepository provides JPA related methods such as flushing, persistence context, and deletes a record in a batch. For example:

```
public interface BookDAO extends JpaRepository
{
}
```

HIBERNATE ASSOCIATION MAPPINGS

one-to-many/many-to-one

one-to-one

many-to-many

Explain by yourself

{CH=12}

Cryptography: *Cryptography* is the art and science of making a cryptosystem that is capable of providing information security. Cryptography aims to provide the following:

- Secret Communication
- Authentic Information
- Store Sensitive Information

Java Cryptography Architecture (JCA) [short note on Java Cryptography Architecture \(5\)](#)

The **Java Cryptography Architecture (JCA)** is based on “provider” architecture. It is a set of APIs to cryptography such as encryption, message digests, key generation and management, digital signatures, certificates etc. It enables you to encrypt and decrypt data in java, manage keys, sign and authenticate messages etc.

JCA provides some general-purpose classes and interfaces. JCA has “provider” based architecture; thus actual functionality and implementation are provided by providers. The encryption algorithm used for encryption and decryption in Cipher class depends on the concrete provider used.

JAVA SECURE SOCKET EXTENSION (JSSE)

The Java Secure Socket Extension (JSSE) is a framework to enable secure internet communication. This framework provides an implementation for a Java version of the Secure Socket Layer (SSL) and Transport Layer Security (TLS) protocols. It also provides API's for data encryption, server authentication, message integrity and optional client Authentication.

JSSE provides API framework along with its implementation. It acts as a building block to build secure web applications simply. It reduces the security vulnerabilities by abstracting the complex underlying security algorithms and handshaking mechanisms. The JSSE API boosts up the core network and cryptographic services defined by the java.security and java.net packages with add-on security.

JSSE Features and Benefits

- A standard component of the JDK
- Provider-based architecture makes it flexible and extensible
- Provides classes to create secure channels such as SSLSocket, SSLServerSocket, and SSLEngine
- Support for client and server authentication, which is part of the normal SSL/TLS/DTLS handshaking
- Server name indication extension to facilitate secure connections to virtual servers.
- Provides support for Server Name Indication (SNI) Extension
- Prevents man-in-the-middle attacks by providing support for endpoint identification during handshaking.

Issues and Challenges of Web Security [←Explain this \(10\)](#)

- 1. Ransomware Evolution:** Ransomware is a type of malware in which the data on a victim's computer is locked, and payment is demanded before the ransomed data is unlocked. After successful payment, access rights returned to the victim. Ransomware is the bane of cybersecurity, data professionals, IT, and executives.
- 2. Blockchain Revolution:** Blockchain technology is the most important invention in computing era. It is the first time in human history that we have a genuinely native digital medium for peer-to-peer value exchange. The blockchain is a technology that enables cryptocurrencies like Bitcoin. The blockchain is a vast global platform that allows two or more parties to do a transaction or do business without needing a third party for establishing trust.
- 3. IoT Threats:** IoT stands for Internet of Things. It is a system of interrelated physical devices which can be accessible through the internet. The connected physical devices have a unique identifier (UID) and have the ability to transfer data over a network without any requirements of the human-to-human or human-to-computer interaction. The firmware and software which is running on IoT devices make consumer and businesses highly susceptible to cyber-attacks.
- 4. AI Expansion:** AI short form is Artificial intelligence. According to John McCarthy, father of Artificial Intelligence defined AI: "The science and engineering of making intelligent machines, especially intelligent computer programs."
It is an area of computer science which is the creation of intelligent machines that do work and react like humans. Some of the activities related to artificial intelligence include speech recognition, Learning, Planning, Problem-solving, etc. The key benefits with AI into our cybersecurity strategy has the ability to protect and defend an environment when the malicious attack begins, thus mitigating the impact. AI take immediate action against the malicious attacks at a moment when a threats impact a business.
- 5. Serverless Apps Vulnerability:** Serverless architecture and apps is an application which depends on third-party cloud infrastructure or on a back-end service such as google cloud function, Amazon web services (AWS) lambda, etc. The serverless apps invite the cyber attackers to spread threats on their system easily because the users access the application locally or off-server on their device. Therefore it is the user responsibility for the security precautions while using serverless application.

-
- **Authentication:** Authentication refers to the process of confirming user's identity whom he claims to be. It's about validating user's credentials such as Email/Username/UserId and password. The system validates whether the user is one whom he claims to be using user's credentials.
 - **Authorization:** Authorization action takes place once a system has authenticated the user's identity. Authorization refers to the rules that determine who is allowed to do what. E.g. John may be authorized to create and delete databases, while Bob is only authorized to read. Authorization is a process of permitting required privileges to the user to access a specific resource in an application such as files, location, database et
-

ROLES BASED LOGIN **short note (5):** Role based login means redirecting users to different URLs upon successful login according to the assigned role to the logged-in user. Role-based access control (RBAC) is a mechanism that restricts system access. RBAC accomplishes this by assigning one or more "roles" to each user and giving each role different permissions

CROSS SITE REQUEST FORGERY (CSRF): CSRF is a type of attack in which attackers trick a victim into making a request that utilizes their authentication or authorization. The victim's level of permission decides the impact of CSRF attack. Execution of CSRF attack consists of mainly two-part.

- Trick the victim into clicking a link or loading a web page
- Sending a crafted, legitimate-looking request from victim's browser to website.

The most popular method to prevent CSRF is to use a randomly generated token that is associated with a particular user and that is sent as a hidden value in every state-changing form in the web app. This token, called an anti-CSRF token (often abbreviated as CSRF token) or a synchronizer token.

Spring Security CSRF Protection: Necessary steps to use Spring Security's CSRF protection are as follows-

Use proper HTTP verb	Configure CSRF protection	Include the CSRF token
----------------------	---------------------------	------------------------